



Prácticas de la asignatura

Cálculo Numérico II

Grado en Matemáticas por la Universidad de Sevilla
Dpto. de Ecuaciones Diferenciales y Análisis Numérico
Universidad de Sevilla

Curso 2017/18

Índice

1. Introducción y elementos básicos	5
1.1. Comenzando	5
1.2. Objetos y sintaxis básicos	5
1.2.1. Constantes y operadores	7
1.2.2. Funciones elementales	7
1.2.3. Uso como calculadora	7
1.2.4. Variables	8
1.2.5. Formatos	9
1.2.6. Algunos comandos utilitarios del sistema operativo	9
1.3. Documentación y ayuda <i>on-line</i>	10
1.4. <i>Scripts</i> y funciones. El editor integrado	10
1.4.1. <i>Scripts</i>	10
1.4.2. M-Funciones	11
1.4.3. Funciones anónimas	12
1.5. <i>Workspace</i> y ámbito de las variables	13
1.6. Matrices	13
1.6.1. Construcción de matrices	13
1.6.2. Acceso a los elementos de vectores y matrices	15
1.6.3. Operaciones con vectores y matrices	16
1.6.4. Resolución de sistemas lineales	18
1.7. Dibujo de curvas	20
1.8. Ejercicios	24
2. Programación con MATLAB	27
2.1. Estructuras condicionales: if	27
2.2. Estructuras de repetición o bucles: while	30
2.3. Estructuras de repetición o bucles indexados: for	32
2.4. Ruptura de bucles de repetición: break y continue	34
2.5. Gestión de errores: warning y error	35
2.6. Operaciones de lectura y escritura	36
2.6.1. Instrucción básica de lectura: input	36

2.6.2. Instrucción básica de impresión en pantalla: <code>disp</code>	36
2.6.3. Instrucción de impresión en pantalla con formato: <code>fprintf</code>	37
2.7. Comentarios generales	38
2.8. Ejercicios	39
3. Resolución de ecuaciones no lineales	43
3.1. Métodos iterados y su programación	43
3.1.1. El método de bisección o dicotomía	43
3.1.2. El método de la falsa posición (<i>regula falsi</i>)	46
3.1.3. El método de aproximaciones sucesivas	48
3.1.4. El método de Newton	50
3.2. Resolución de ecuaciones no lineales con MATLAB	53
3.2.1. La función <i>fzero</i>	53
3.2.2. Gráficas para localizar las raíces y elegir el punto inicial	54
4. Integración numérica	57
4.1. La función <i>integral</i> (<i>quad</i> en versiones antiguas de MATLAB)	57
4.2. La función <i>trapz</i> (fórmula de los trapecios)	61
5. Interpolación y ajuste polinómico	63
5.1. Interpolación global ó de Lagrange	63
5.2. Interpolación lineal a trozos	65
5.3. Interpolación mediante <i>splines</i> cúbicos	66
5.4. Ajuste de datos	70
6. Resolución de ecuaciones y sistemas diferenciales ordinarios	75
6.1. Resolución de problemas de Cauchy con MATLAB	76

1 Introducción y elementos básicos

MATLAB es un potente paquete de software para computación científica, orientado al cálculo numérico, a las operaciones matriciales y especialmente a las aplicaciones científicas y de ingeniería. Ofrece lo que se llama un entorno de desarrollo integrado (IDE), es decir, una herramienta que permite, en una sola aplicación, ejecutar órdenes sencillas, escribir programas utilizando un editor integrado, compilarlos (o interpretarlos), depurarlos (buscar errores) y realizar gráficas.

Puede ser utilizado como simple calculadora matricial, pero su interés principal radica en los cientos de funciones tanto de propósito general como especializadas que posee, así como en sus posibilidades para la visualización gráfica.

MATLAB posee un lenguaje de programación propio, muy próximo a los habituales en cálculo numérico (Fortran, C, ...), aunque mucho más *tolerante* en su sintaxis, que permite al usuario escribir sus propios *scripts* (conjunto de comandos escritos en un fichero, que se pueden ejecutar con una única orden) para resolver un problema concreto y también escribir nuevas funciones con, por ejemplo, sus propios algoritmos, o para *modularizar* la resolución de un problema complejo. MATLAB dispone, además, de numerosas *Toolboxes*, que le añaden funcionalidades especializadas.

Numerosas contribuciones de sus miles de usuarios en todo el mundo pueden encontrarse en la web de The MathWorks: <http://www.mathworks.es>

1.1 Comenzando

Al iniciar MATLAB nos aparecerá una ventana más o menos como la de la Figura 1.1 (dependiendo del sistema operativo y de la versión)

Si la ubicación de las ventanas integradas es diferente, se puede volver a ésta mediante:

Menú Desktop → Desktop Layout → Default

Se puede experimentar con otras disposiciones. Si hay alguna que nos gusta, se puede salvaguardar con

Menú Desktop → Desktop Layout → Save Layout ...

dándole un nombre, para usarla en otras ocasiones. De momento, sólo vamos a utilizar la ventana principal de MATLAB: **Command Window** (ventana de comandos). A través de esta ventana nos comunicaremos con MATLAB, escribiendo las órdenes en la línea de comandos. El símbolo **>>** al comienzo de una línea de la ventana de comandos se denomina *prompt* e indica que MATLAB está desocupado, disponible para ejecutar nuestras órdenes.

1.2 Objetos y sintaxis básicos

Las explicaciones sobre las funciones/comandos que se presentan en estas notas están muy resumidas y sólo incluyen las funcionalidades que, según el parecer subjetivo de la autora, pueden despertar más

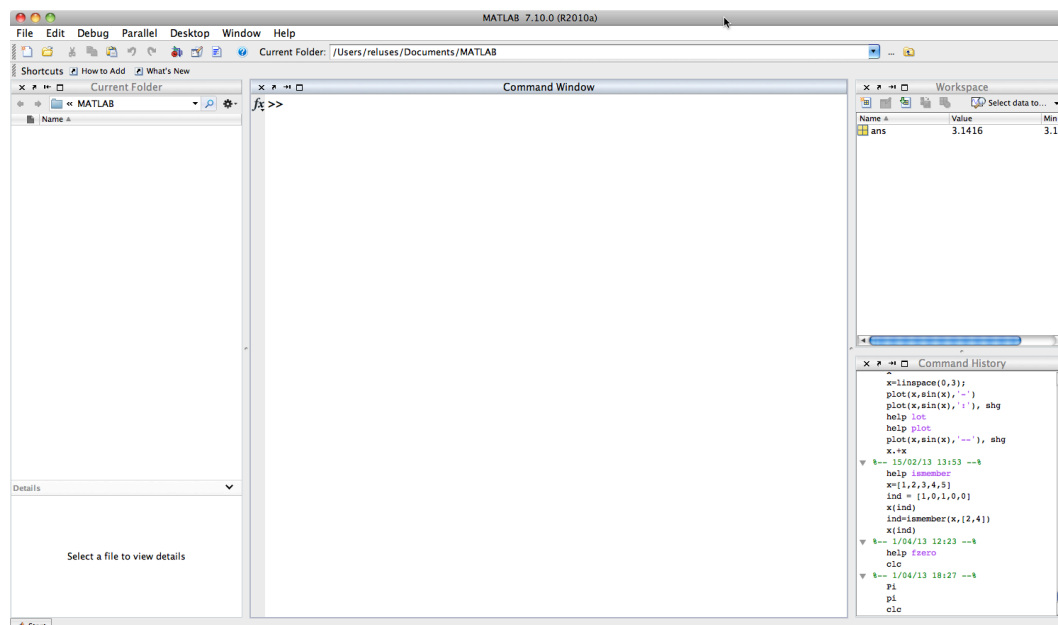


Figura 1.1: La ventana de MATLAB

interés. La mayoría de las funciones tienen mas y/o distintas funcionalidades que las que se exponen aquí. Para una descripción exacta y exhaustiva es preciso consultar la Ayuda on-line.

Los tipos básicos de datos que maneja MATLAB son números reales, booleanos (valores lógicos) y cadenas de caracteres (string). También puede manipular distintos tipos de números enteros, aunque sólo suele ser necesario en circunstancias específicas.

En MATLAB, por defecto, los números son codificados como números reales en coma flotante en doble precisión. La precisión, esto es, el número de bits dedicados a representar la mantisa y el exponente, depende de cada (tipo de) máquina.

MATLAB manipula también otros objetos, compuestos a partir de los anteriores: números complejos, matrices, *cells*, estructuras definidas por el usuario, clases Java, etc.

El objeto básico de trabajo de MATLAB es una matriz bidimensional cuyos elementos son números reales o complejos. Escalares y vectores son considerados casos particulares de matrices. También se pueden manipular matrices de cadenas de caracteres, booleanas y enteras.

El lenguaje de MATLAB es **interpretado**, esto es, las instrucciones se traducen a lenguaje máquina una a una y se ejecutan antes de pasar a la siguiente. Es posible escribir varias instrucciones en la misma línea, separándolas por una coma o por punto y coma. Las intrucciones que terminan por punto y coma no producen salida de resultados por pantalla.

Algunas constantes numéricas están predefinidas (ver Tabla 1.1)

MATLAB distingue entre mayúsculas y minúsculas: **pi** no es lo mismo que **Pi**.

MATLAB conserva un historial de las instrucciones escritas en la línea de comandos. Se pueden recuperar instrucciones anteriores, usando las teclas de flechas arriba y abajo. Con las flechas izquierda y derecha nos podemos desplazar sobre la línea de comando y modificarlo.

Se pueden salvaguardar todas las instrucciones y la salida de resultados de una sesión de trabajo de MATLAB a un fichero:

```
>> diary nombre_fichero
>> diary off % suspende la salvaguarda
```

<code>i, j</code>	Unidad imaginaria : <code>2+3i, -1-2j</code>
<code>pi</code>	Número π
<code>Inf</code>	<i>Infinito</i> , número mayor que el más grande que se puede almacenar. Se produce con operaciones como $x/0$, con $x \neq 0$
<code>NaN</code>	<i>Not a Number</i> , magnitud no numérica resultado de cálculos indefinidos. Se produce con cálculos del tipo $0/0$ o ∞/∞ . <code>(0+2i)/0</code> da como resultado <code>NaN + Inf i</code>
<code>eps, intmax, intmin</code> <code>realmax, realmin</code>	Otras constantes. Consultar la ayuda on-line.

Tabla 1.1: Algunas constantes pre-definidas en MATLAB. Sus nombres son reservados y no deberían ser usados para variables ordinarias.

1.2.1 Constantes y operadores

Números reales	<code>8.01 -5.2 .056 1.4e+5 0.23E-2</code> <code>-.567d-21 8.003D-12</code>
Números complejos	<code>1+2i -pi-3j</code>
Booleanos	<code>true false</code>
Caracteres	Entre apóstrofes: <code>'esto es una cadena de caracteres (string)'</code>
Operadores aritméticos	<code>+ - * / ^</code>
Operadores de comparación	<code>== ~= (ó <>) < > <= ></code>
Operadores lógicos (los dos últimos sólo para escalares)	<code>& ~ && </code> <code>&& y </code> no evalúan el operando de la derecha si no es necesario.)

Tabla 1.2: Constantes y operadores de diversos tipos.

1.2.2 Funciones elementales

Los nombres de las funciones elementales son los *habituales*.

Los argumentos pueden ser, siempre que tenga sentido, reales o complejos y el resultado se devuelve en el mismo tipo del argumento.

La lista de todas las funciones matemáticas elementales se puede consultar en:

[Help](#) → [MATLAB](#) → [Functions: By Category](#) → [Mathematics](#) → [Elementary Math](#)

Algunas de las más habituales se muestran en la Tabla 1.3:

1.2.3 Uso como calculadora

Se puede utilizar MATLAB como simple calculadora, escribiendo expresiones aritméticas y terminando por RETURN (`<R>`). Se obtiene el resultado inmediatamente a través de la variable del sistema `ans` (de *answer*). Si no se desea que MATLAB escriba el resultado en el terminal, debe terminarse la orden por punto y coma (útil, sobre todo en programación).

<code>sqrt(x)</code>	raíz cuadrada	<code>sin(x)</code>	seno (radianes)
<code>abs(x)</code>	módulo	<code>cos(x)</code>	coseno (radianes)
<code>conj(z)</code>	complejo conjugado	<code>tan(z)</code>	tangente (radianes)
<code>real(z)</code>	parte real	<code>cotg(x)</code>	cotangente (radianes)
<code>imag(z)</code>	parte imaginaria	<code>asin(x)</code>	arcoseno
<code>exp(x)</code>	exponencial	<code>acos(x)</code>	arcocoseno
<code>log(x)</code>	logaritmo natural	<code>atan(x)</code>	arcotangente
<code>log10(x)</code>	logaritmo decimal	<code>cosh(x)</code>	cos. hiperbólico
<code>rat(x)</code>	aprox. racional	<code>sinh(x)</code>	seno hiperbólico
<code>mod(x,y)</code> <code>rem(x,y)</code>	resto de dividir x por y Iguales si $x, y > 0$. Ver help para definición exacta	<code>tanh(x)</code>	tangente hiperbólica
<code>fix(x)</code>	Redondeo hacia 0	<code>acosh(x)</code>	arcocoseno hiperb.
<code>ceil(x)</code>	Redondeo hacia $+\infty$	<code>asinh(x)</code>	arcoseno hiperb.
<code>floor(x)</code>	Redondeo hacia $-\infty$	<code>atanh(x)</code>	arcotangente hiperb.
<code>round(x)</code>	Redondeo al entero más próximo		

Tabla 1.3: Algunas funciones matemáticas elementales.

Ejemplos 1.1

```
>> sqrt(34*exp(2))/(cos(23.7)+12)
ans =
    1.3058717

>> 7*exp(5/4)+3.54
ans =
    27.97240

>> exp(1+3i)
ans =
    - 2.6910786 + 0.3836040i
```

1.2.4 Variables

En MATLAB las variables no son nunca declaradas: su tipo y su tamaño cambian de forma dinámica de acuerdo con los valores que le son asignados. Así, una misma variable puede ser utilizada, por ejemplo, para almacenar un número complejo, a continuación una matriz 25×40 de números enteros y luego para almacenar un texto. Las variables se crean automáticamente al asignarles un contenido. Asimismo, es posible eliminar una variable de la memoria si ya no se utiliza.

Ejemplos 1.2

```
>> a=10
    a =
    10.
>> pepito=exp(2.4/3)
    pepito =
    2.2255
>> pepito=a+pepito*(4-0.5i)
    pepito =
    18.9022 - 1.1128i
>> clear pepito
```

Para conocer en cualquier instante el valor almacenado en una variable basta con teclear su nombre (Atención: recuérdese que las variables **AB**, **ab**, **Ab** y **aB** **son distintas**, ya que MATLAB distingue entre mayúsculas y minúsculas).

Otra posibilidad es hojear el **Workspace** ó espacio de trabajo, abriendo la ventana correspondiente. Ello nos permite ver el contenido de todas las variables existentes en cada momento e, incluso, modificar su valor.

Algunos comandos relacionados con la inspección y eliminación de variables se describen en la tabla siguiente:

who	lista las variables actuales
whos	como el anterior, pero más detallado
clear	elimina todas las variables que existan en ese momento
clear a b c	elimina las variables a , b y c (atención: sin comas!)

1.2.5 Formatos

Por defecto, MATLAB muestra los números en formato de punto fijo con 5 dígitos. Se puede modificar este comportamiento mediante el comando **format**

format	Cambia el formato de salida a su valor por defecto, short
format short	El formato por defecto
format long	Muestra 15 dígitos
format short e	Formato short, en coma flotante
format long e	Formato long, en coma flotante
format rat	Muestra los números como cociente de enteros

1.2.6 Algunos comandos utilitarios del sistema operativo

Están disponibles algunos comandos utilitarios, como

ls dir	Lista de ficheros del directorio de trabajo
pwd	Devuelve el nombre y la ruta (<i>path</i>) del directorio de trabajo
cd	Para cambiar de directorio
clc	Limpia la ventana de comandos Command Window
date	Fecha actual

Tabla 1.4: Comandos utilitarios.

1.3 Documentación y ayuda *on-line*

- Ayuda on-line en la ventana de comandos

```
>> help nombre_de_comando
```

La información se obtiene en la misma ventana de comandos. Atención:

- Ayuda on-line con ventana de navegador

```
>> helpwin
```

ó bien **Menú Help** ó bien **Botón Start → Help**.

Además, a través del navegador del **Help** se pueden descargar, desde The MathWorks, guías detalladas, en formato pdf, de cada capítulo.

1.4 *Scripts* y funciones. El editor integrado

1.4.1 *Scripts*

En términos generales, en informática, un *script* (guión o archivo por lotes) es un conjunto de instrucciones (programa), usualmente simple, guardadas en un fichero (usualmente de texto plano) que son ejecutadas normalmente mediante un intérprete. Son útiles para automatizar pequeñas tareas. También puede hacer las veces de un "programa principal" para ejecutar una aplicación.

Así, para llevar a cabo una tarea, en vez de escribir las instrucciones una por una en la línea de comandos de MATLAB, se pueden escribir las órdenes una detrás de otra en un fichero. Para ello se puede utilizar el **Editor integrado de MATLAB**. Para iniciarlo, basta pulsar el icono *hoja en blanco* (**New script**) de la barra de MATLAB, o bien

File → New → Script

UN *script* de MATLAB debe guardarse en un fichero con sufijo **.m** para ser reconocido. El nombre del fichero puede ser cualquiera *razonable*, es decir, sin acentos, sin espacios en blanco y sin caracteres «extraños».

Para editar un *script* ya existente, basta hacer *doble-click* sobre su icono en la ventana **Current Folder**. Para ejecutar un *script* que esté en el directorio de trabajo, basta escribir su nombre (sin el sufijo) en la línea de comandos.

1.4.2 M-Funciones

Una función (habitualmente denominadas M-funciones en MATLAB), es un programa con una «interfaz» de comunicación con el exterior mediante argumentos de entrada y de salida.

Las funciones MATLAB responden al siguiente formato de escritura:

```
function [argumentos de salida] = nombre(argumentos de entrada)
%
% comentarios
%
....
instrucciones (normalmente terminadas por ; para evitar eco en pantalla)
....
```

Las funciones deben guardarse en un fichero con el mismo nombre que la función y sufijo `.m`. Lo que se escribe en cualquier línea detrás de `%` es considerado como comentario.

Ejemplo 1.3

El siguiente código debe guardarse en un fichero de nombre `areaequi.m`.

```
function [sup] = areaequi(long)
%
% areaequi(long) devuelve el area del triangulo
%           equilatero de lado = long
%
>> sup = sqrt(3)*long^2/4;
```

La primera línea de una M-función siempre debe comenzar con la cláusula (palabra reservada) `function`. El fichero que contiene la función debe estar *en un sitio en el que MATLAB lo pueda encontrar*, normalmente, en la carpeta de trabajo.

Se puede ejecutar la M-función en la misma forma que cualquier otra función de MATLAB:

Ejemplos 1.4

```
>> areaequi(1.5)
ans =
    0.9743
>> rho = 4 * areaequi(2) +1;
>> sqrt(areaequi(rho))
```

Los breves comentarios que se incluyen a continuación de la línea que contiene la cláusula `function` deben explicar, brevemente, el funcionamiento y uso de la función. Además, constituyen la ayuda *on-line* de la función:

Ejemplo 1.5

```
>> help areaequi
    areaequi(long) devuelve el area del triangulo
    equilatero de lado = long
```

Se pueden incluir en el mismo fichero otras funciones, denominadas subfunciones, a continuación de la primera¹, pero sólo serán *visibles* para las funciones del mismo fichero. **NO** se pueden incluir M-funciones en el fichero de un *script*.

1.4.3 Funciones anónimas

Algunas funciones *sencillas*, que devuelvan el resultado de una expresión, se pueden definir mediante una sola instrucción, en mitad de un programa (script o función) o en la línea de comandos. Se llaman funciones anónimas. La sintaxis para definir las es:

```
nombre_funcion = @(argumentos) expresion_funcion
```

Ejemplo 1.6 (Función anónima para calcular el área de un círculo)

```
>> area_circulo = @(r) pi * r.^2;
>> area_circulo(1)
ans =
    3.1416
>> semicirc = area_circulo(3)/2
semicirc =
    14.1372
```

Las funciones anónimas pueden tener varios argumentos y hacer uso de variables previamente definidas:

Ejemplo 1.7 (Función anónima de dos variables)

```
>> a = 2;
>> mifun = @(x,t) sin(a*t).*cos(t/a);
>> mifun(pi/4,1)
ans =
    0.7980
```

Si, con posterioridad a la definición de la función `mifun`, se cambia el valor de la variable `a`, la función no se modifica: en el caso del ejemplo, seguirá siendo `mifun(x,t)=sin(2*t).*cos(t/2)`.

¹También es posible definir funciones anidadas, esto es, funciones «insertadas» dentro del código de otras funciones. (Se informa aquí para conocer su existencia. Su utilización es delicada.)

1.5 Workspace y ámbito de las variables

Workspace (espacio de trabajo) es el conjunto de variables que en un momento dado están definidas en la memoria del MATLAB.

Las variables creadas desde la línea de comandos de MATLAB pertenecen al **Base Workspace** (espacio de trabajo base; es el que se puede «hojear» en la ventana **Workspace**). Lo mismo sucede con las variables creadas por un *script* que se ejecuta desde la línea de comandos. Estas variables permanecen en el **Base Workspace** cuando se termina la ejecución del script y se mantienen allí durante toda la sesión de trabajo o hasta que se borren.

Sin embargo, las variables creadas por una M-función pertenecen al espacio de trabajo de dicha función, que es independiente del espacio de trabajo base. Es decir, las variables de las M-funciones son **locales**: MATLAB reserva una zona de memoria cuando comienza a ejecutar una M-función, almacena en esa zona las variables que se crean dentro de ella, y «borra» dicha zona cuando termina la ejecución de la función.

Esta es una de las principales diferencias entre un *script* y una M-función: cuando finaliza la ejecución de un *script* se puede «ver» y utilizar el valor de todas las variables que ha creado el script en el **Workspace**; en cambio, cuando finaliza una función no hay rastro de sus variables en el **Workspace**.

Para hacer que una variable local de una función pertenezca al **Base Workspace**, hay que declararla **global**: la orden

```
global a suma error
```

en una función hace que las variables **a**, **suma** y **error** pertenezcan al **Base Workspace** y por lo tanto, seguirán estando disponibles cuando finalice la ejecución de la función.

1.6 Matrices

Como ya se ha dicho, las matrices bidimensionales de números reales o complejos son los objetos básicos con los que trabaja MATLAB. Los vectores y escalares son casos particulares de matrices.

1.6.1 Construcción de matrices

La forma más elemental de introducir matrices en MATLAB es describir sus elementos de forma exhaustiva (por filas y entre corchetes rectos **[]**): elementos de una fila se separan unos de otros por comas y una fila de la siguiente por punto y coma.

Ejemplos 1.8 (Construcciones elementales de matrices)

```
>> v = [1, -1, 0, sin(2.88)]           % vector fila longitud 4

>> w = [0; 1.003; 2; 3; 4; 5*pi]       % vector columna longitud 6

>> a = [1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12] % matriz 3x4
```

Observación. El hecho de que, al introducirlas, se escriban las matrices por filas no significa que internamente, en la memoria del ordenador, estén así organizadas: **en la memoria las matrices se almacenan como un vector unidimensional ordenadas por columnas.**

Ejemplos 1.9 (Otras órdenes para crear matrices)

```
>> v1 = a:h:b      % crea un vector fila de números regularmente espaciados
                    % a a+h a+2h ... hasta c <= b < c+h

>> v2 = a:b        % como el anterior, con paso h=1

>> v3 = v2'        % matriz traspuesta (conjugada si es compleja)

>> v4 = v2.'       % matriz traspuesta sin conjugar
```

Se pueden también utilizar los vectores/matrices como objetos para construir otras matrices (bloques):

Ejemplos 1.10 (Matrices construidas con bloques)

```
>> v1 = 1:4;

>> v2 = [v1, 5; 0.1:0.1:0.5]

>> v3 = [v2', [11,12,13,14,15]']
```

$$v_1 = \begin{pmatrix} 1 & 2 & 3 & 4 \end{pmatrix} \quad v_2 = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 0.1 & 0.2 & 0.3 & 0.4 & 0.5 \end{pmatrix} \quad v_3 = \begin{pmatrix} 1 & 0.1 & 11 \\ 2 & 0.2 & 12 \\ 3 & 0.3 & 13 \\ 4 & 0.4 & 14 \\ 4 & 0.5 & 15 \end{pmatrix}$$

Las siguientes funciones generan vectores de elementos regularmente espaciados, útiles en muchas circunstancias, especialmente para creación de gráficas.

<code>linspace(a,b,n)</code>	Si a y b son números reales y n un número entero, genera una partición regular del intervalo [a,b] con n nodos (n-1 subintervalos)
<code>linspace(a,b)</code>	Como el anterior, con n=100

Las siguientes funciones generan algunas matrices especiales que serán de utilidad.

<code>zeros(n,m)</code>	matriz $n \times m$ con todas sus componentes iguales a cero
<code>ones(n,m)</code>	matriz $n \times m$ con todas sus componentes iguales a uno
<code>eye(n,m)</code>	matriz unidad $n \times m$: diagonal principal = 1 y el resto de las componentes = 0
<code>diag(v)</code>	Si v es un vector, es una matriz cuadrada de ceros con diagonal principal = v
<code>diag(A)</code>	Si A es una matriz, es su diagonal principal

MATLAB posee, además, decenas de funciones útiles para generar distintos tipos de matrices. Para ver una lista exhaustiva consultar:

[Help](#) → [MATLAB](#) → [Functions: By Category](#) → [Mathematics](#) → [Arrays and Matrices](#)

1.6.2 Acceso a los elementos de vectores y matrices

El acceso a elementos individuales de una matriz se hace indicando entre paréntesis sus índices de fila y columna. Se puede así utilizar o modificar su valor

Ejemplos 1.11 (Acceso a elementos individuales de una matriz)

```
>> A = [1, 2, 3; 6, 3, 1; -1, 0, 0; 2, 0, 4]
```

```
A =
```

```

1     2     3
6     3     1
-1    0     0
2     0     4
```

```
>> A(2, 3)
```

```
ans =
```

```
1
```

```
>> A(4, 2) = 11
```

```
A =
```

```

1     2     3
6     3     1
-1    0     0
2    11     4
```

Se puede acceder al último elemento de un vector mediante el *índice simbólico* `end`:

Ejemplos 1.12 (Último índice de un vector)

```
>> w = [5, 3, 1, 7, 3, 0, -2];
```

```
>> w(end)
```

```
ans =
```

```
-2
```

```
>> w(end-1) = 101
```

```
w =
```

```

5     3     1     7     3    101    -2
```

Se puede acceder a una fila o columna completa sustituyendo el índice adecuado por `:` (dos puntos)

Ejemplos 1.13 (Acceso a filas o columnas completas)

```
>> A = [1, 2, 3; 6, 3, 1; -1, 0, 0; 2, 0, 4];
>> A(:, 3)
ans =
     3
     1
     0
     4
>> A(2, :)
ans =
     6     3     1
>> A(3, :) = [5, 5, 5]
A =
     1     2     3
     6     3     1
     5     5     5
     2     0     4
```

También se puede acceder a una “submatriz” especificando los valores de las filas y columnas que la definen:

Ejemplos 1.14 (Acceso a “submatrices”)

```
>> w = [5, 3, 1, 7, 3, 0, -2];
>> w([1,2,3])           % o lo que es lo mismo w(1:3)
ans =
     5     3     1
>> A = [1, 2, 3; 6, 3, 1; -1, 0, 0; 2, 0, 4];
>> A(1:2, :) = zeros(2, 3)
A =
     0     0     0
     0     0     0
    -1     0     0
     2     0     4
>> B = A(3:4, [1,3])
B =
    -1     0
     2     4
```

1.6.3 Operaciones con vectores y matrices

Los operadores aritméticos representan las correspondientes operaciones matriciales siempre que tengan sentido.

Sean A y B dos matrices de elementos respectivos a_{ij} y b_{ij} y sea k un escalar.	
A+B , A-B	matrices de elementos respectivos $a_{ij} + b_{ij}$, $a_{ij} - b_{ij}$ (si las dimensiones son iguales)
A+k , A-k	matrices de elementos respectivos $a_{ij} + k$, $a_{ij} - k$.
k*A , A/k	matrices de elementos respectivos $k a_{ij}$, $\frac{1}{k} a_{ij}$
A*B	producto matricial de A y B (si las dimensiones son adecuadas)
A^n	Si n es un entero positivo, A*A*...*A

Además de estos operadores, MATLAB dispone de ciertos operadores aritméticos que operan **elemento a elemento**. Son los operadores **.***, **./** y **.^**, muy útiles para aprovechar las funcionalidades vectoriales de MATLAB.

Sean A y B dos matrices de las mismas dimensiones y de elementos respectivos a_{ij} y b_{ij} y sea k un escalar.	
A.*B	matriz de la misma dimensión que A y B de elementos $a_{ij} \times b_{ij}$
A./B	ídem de elementos $\frac{a_{ij}}{b_{ij}}$
A.^B	ídem de elementos $a_{ij}^{b_{ij}}$
k./A	matriz de la misma dimensión que A , de elementos $\frac{k}{a_{ij}}$
A.^k	ídem de elementos a_{ij}^k
k.^A	ídem de elementos $k^{a_{ij}}$

Por otra parte, la mayoría de las funciones MATLAB están hechas de forma que admiten matrices como argumentos. Esto se aplica en particular a las funciones matemáticas elementales y su utilización debe entenderse en el sentido de **elemento a elemento**. Por ejemplo, si **A** es una matriz de elementos a_{ij} , **exp(A)** es otra matriz con las mismas dimensiones que **A**, cuyos elementos son $e^{a_{ij}}$.

Algunas otras funciones útiles en cálculos matriciales son:

<code>sum(v)</code>	suma de los elementos del vector v
<code>sum(A)</code> <code>sum(A,1)</code>	suma de los elementos de la matriz A , por columnas
<code>sum(A,2)</code>	suma de los elementos de la matriz A , por filas
<code>prod(v)</code> , <code>prod(A)</code> <code>prod(A,1)</code> <code>prod(A,2)</code>	como la suma, pero para el producto
<code>max(v)</code> , <code>min(v)</code>	máximo/mínimo elemento del vector v
<code>max(A)</code> , <code>min(A)</code>	máximo/mínimo elemento de la matriz A , por columnas
<code>mean(v)</code>	promedio de los elementos del vector v
<code>mean(A)</code>	promedio de los elementos de la matriz A , por columnas.

1.6.4 Resolución de sistemas lineales

Sea A una matriz cualquiera y sea b un vector columna con el mismo número de filas que A . Entonces, la solución del sistema

$$Ax = b$$

se calcula en MATLAB mediante el operador no estándar `\` denominado *backward slash*, (barra inversa en español)

```
x = A \ b
x = mldivide(A, b)      (equivalente a lo anterior)
```

Hay que pensar en él como el operador de “división matricial por la izquierda”.

Este operador se aplica incluso si A no es una matriz cuadrada, es decir si no hay el mismo número de ecuaciones que de incógnitas. Lo que se obtiene de estas operaciones es, lógicamente, distinto según sea el caso.

De forma resumida, si A es una matriz cuadrada y b es un vector columna, $x = A \backslash b$ es la solución del sistema lineal $Ax = b$, obtenida por diferentes algoritmos, en función de las características de la matriz A (diagonal, triangular, simétrica, etc.). Si A es singular o mal condicionada, se obtendrá un mensaje de alerta (*warning*).

Ejemplo 1.15 (Uso del operador `\`)

Calcular la solución del sistema (compatible determinado)

$$\begin{cases} 2x_1 + x_2 - x_3 &= -1 \\ 2x_1 - x_2 + 3x_3 &= -2 \\ 3x_1 - 2x_2 &= 1 \end{cases}$$

```
>> A = [2, 1, -1; 2, -1, 3; 3, -2, 0];
>> b = [-1; -2; 1];
>> x = A\b
x =
    -0.3636
    -1.0455
    -0.7727
```

Como comprobación calculamos el residuo que, como es de esperar, no es exactamente nulo, debido a los errores de redondeo:

```
>> A*x - b
ans =
    1.0e-15 *
    -0.4441
         0
         0
```

Ejemplo 1.16 (Uso del operador \)

Calcular la solución del sistema (compatible indeterminado)

$$\begin{cases} x_1 + x_2 + x_3 &= 1 \\ 2x_1 - x_2 + x_3 &= 2 \\ x_1 - 2x_2 &= 1 \end{cases}$$

```
>> A = [1, 1, 1; 2, -1, 1; 1, -2, 0];
>> b = [ 1; 2; 1];
>> x = A\b
Warning: Matrix is singular to working precision.
x =
    NaN
    NaN
    NaN
```

Ejemplo 1.17 (Uso del operador \)

Calcular la solución del sistema (incompatible)

$$\begin{cases} 2x + 2y + t &= 1 \\ 2x - 2y + z &= -2 \\ x - z + t &= 0 \\ -4x + 4y - 2z &= 1 \end{cases}$$

```
>> A = [2, 2, 0, 1; 2, -2, 1, 0; 1, 0, -1, 1; -4, 4, -2, 0];
>> b = [ 1; -2; 0; 1];
>> x = A\b
Warning: Matrix is singular to working precision.
x =
    NaN
    NaN
   -Inf
   -Inf
```

1.7 Dibujo de curvas

La representación gráfica de una curva en un ordenador es una línea poligonal construida uniendo mediante segmentos rectos un conjunto discreto y ordenado de puntos: $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$.

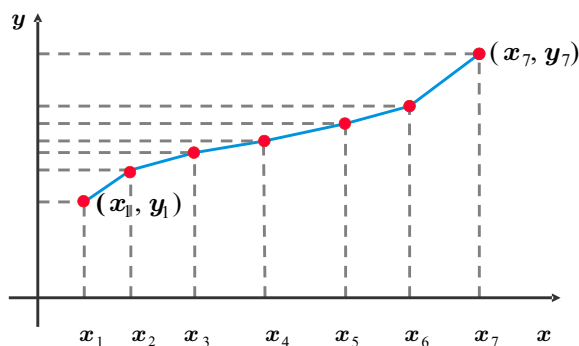


Figura 1.2: Línea poligonal determinada por un conjunto de puntos.

La línea así obtenida tendrá mayor apariencia de «suave» cuanto más puntos se utilicen para construirla, ya que los segmentos serán imperceptibles.

Para dibujar una curva plana en MATLAB se usa el comando

```
plot(x,y)
```

siendo **x** e **y** dos vectores de las mismas dimensiones conteniendo, respectivamente, las abscisas y las ordenadas de los puntos de la gráfica.

Ejemplo 1.18

Dibujar la curva $y = \frac{x^2 + 2}{x + 5}$ para $x \in [-2, 3]$

```
>> f = @(x) (x.^2+2)./(x+5);
>> x = linspace(-2,3);
>> plot(x,f(x))
```

Se pueden dibujar dos o más curvas de una sola vez, proporcionando al comando **plot** varios pares de vectores abscisas-ordenadas, como en el ejemplo siguiente.

Ejemplo 1.19

Dibujar las curvas $y = 2 \sin^3(x) \cos^2(x)$ e $y = e^x - 2x - 3$ para $x \in [-1.5, 1.5]$

```
>> f1 = @(x) 2 * sin(x).^3 .* cos(x).^2;
>> f2 = @(x) exp(x) - 2*x -3;
>> x = linspace(-1.5,1.5);
>> plot(x,f1(x),x,f2(x))
```

A cada par de vectores abscisas-ordenadas en el comando **plot** se puede añadir un argumento opcional de tipo cadena de caracteres, que modifica el aspecto con que se dibuja la curva. Para más información, hojear el help (**help plot**).

Ejemplo 1.20

Las siguientes órdenes dibujan la curva $y = \sin^3(x)$ en color rojo (**r**, de red) y con marcadores *****, en lugar de una línea continua.

```
>> x = linspace(0,pi,30);
>> plot(x,sin(x).^3,'r*')
```

La orden siguiente dibuja la curva $y = \sin^3 x$ en color negro (**k**, de black), con marcadores ***** y con línea punteada, y la curva $y = \cos^2 x$ en color azul (**b**, de blue) y con marcadores **+**

```
>> plot(x,sin(x).^3,'k*:', x, cos(x).^2,'b+')
```

Ademas, mediante argumentos opcionales, es posible modificar muchas otras propiedades de las curvas. Esto se hace siempre mediante un par de argumentos en la orden de dibujo que indican

'Nombre de la Propiedad', Valor de la propiedad

Esta propiedad afectará a todas las curvas que se dibujen con la misma orden.

Ejemplo 1.21

Para establecer un grosor determinado de las líneas se usa la propiedad `LineWidth` (atención a las mayúsculas):

```
>> plot(x,sin(x).^3,'k*:', x, cos(x).^2,'b+', 'LineWidth', 1.1)
```

Se pueden añadir elementos a la gráfica, para ayudar a su comprensión. Para añadir una leyenda que identifique cada curva se usa el comando siguiente, que asigna las leyendas a las curvas en el orden en que han sido dibujadas.

```
legend('Leyenda1', 'Leyenda2')
```

Para añadir etiquetas a los ejes que aclaren el significado de las variables se usan los comandos

```
xlabel('Etiqueta del eje OX')  
ylabel('Etiqueta del eje OY')
```

Se puede añadir una cuadrícula mediante la orden

```
grid on
```

También es muy útil la orden, siguiente, que define las coordenadas mínimas y máxima del rectángulo del plano OXY que se visualiza en la gráfica.

```
axis([xmin, xmax, ymin, ymax])
```

Cada nueva orden de dibujo borra el contenido previo de la ventana gráfica, si existe. Para evitar esto existen la órdenes

```
hold on  
hold off
```

La orden `hold on` permanece activa hasta que se cierre la ventana gráfica o bien se dé la orden `hold off`

Ejemplos 1.22

Las siguientes órdenes darán como resultado la gráfica de la figura 1.3

```
x = linspace(0,pi,30);
axis([-0.5,pi+0.5,-0.5,1.5])
hold on
plot(x,sin(x).^3,'g', x, cos(x).^2,'b+', 'LineWidth', 1.1)
x = linspace(-0.95, pi);
plot(x, log(x+1)/2, 'r', 'LineWidth', 1.1)
plot([-5,5], [0, 0], 'k', 'LineWidth', 1)
plot([0, 0], [-5,5], 'k', 'LineWidth', 1)
legend('Leyenda1', 'Leyenda2', 'Leyenda3')
xlabel('Etiqueta del eje OX')
ylabel('Etiqueta del eje OY')
hold off
shg
```

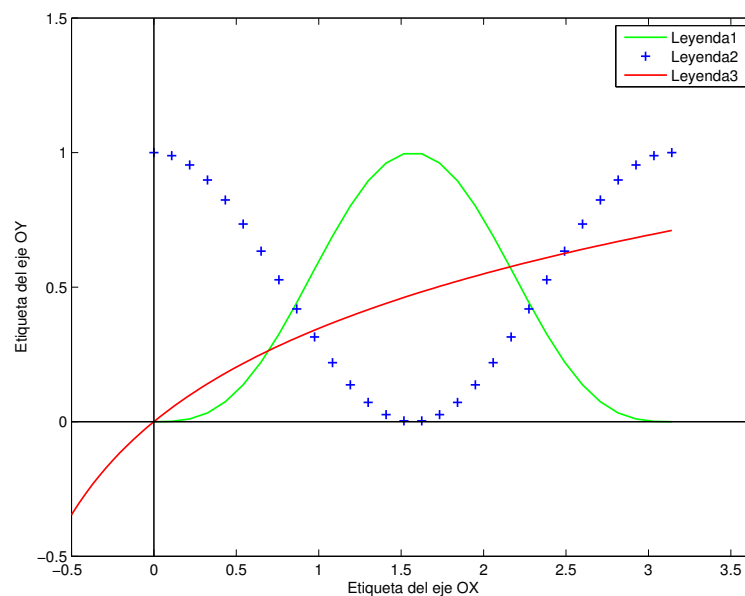


Figura 1.3: Varias curvas con leyenda y etiquetas.

1.8 Ejercicios

- Sean dos vectores-fila $x = (4, 6, 2)$, $y = (3, -2, 4)$ y $k = 3$.
 - Comprobar que: $x + y = (7, 4, 6)$; $x - y = (1, 8, -2)$.
 - Calcular $y - k$; $y + k$; ky ; x/k ; $2x - y$.
 - Realizar las siguientes cálculos componente a componente: $x^2 + y^2$; $\frac{3}{\sqrt[3]{x}}$; $\frac{1+x}{y^4}$.
- Generar los vectores que se indican utilizando los operadores adecuados de MATLAB (esto es, sin escribir expresamente todas sus componentes):
 - $v1 = (1, 3, 5, \dots, 25)$
 - $v2 = (\pi, 2\pi, 3\pi, \dots, 10\pi)$
 - $v3 = (0, 0.1, 0.2, \dots, 1)$
 - $v4 = (-10, -9, -8, \dots, -1, 0)$
- Almacenar, en una variable de nombre n , un número entero par $n \geq 4$. Generar los vectores de longitud n variable que se indican:

(3.a) $w1 = (1, 3, 5, 7, \dots)$	(3.d) $w4 = (2n, 2n - 2, \dots, 4, 2)$
(3.b) $w2 = (\pi, 2\pi, 3\pi, 4\pi, \dots)$	(3.e) $w5 = (0, 2, 2, \dots, 2, 2, 0)$
(3.c) $w3 = (n, n - 1, n - 2, \dots, 2, 1)$	(3.f) $w6 = (1, 3, 5, \dots, n - 1, n, n - 2, \dots, 4, 2)$
- Generar un vector x con 30 componentes regularmente espaciadas entre 0 y π . Evaluar en x cada una de las funciones siguientes:

(4.a) $f(x) = \frac{\cos(x/4)}{\ln(2 + x^2)}$	(4.c) $f(x) = \frac{x}{\ln(2 + x)}$
(4.b) $f(x) = e^{(-x^2+2)} \sin(x/2)$	(4.d) $f(x) = \frac{1}{e^x} + x^2 - x$
- Definir funciones anónimas para calcular las funciones siguientes. Debe hacerse de forma que x pueda ser un vector.

(5.a) $f(x) = \sqrt{\frac{1-x}{1+x}}$	(5.e) $f(x) = \frac{1}{x}$
(5.b) $f(x) = \frac{x}{\ln(x)}$	(5.f) $f(x) = \ln\left(\frac{1}{x-2}\right)$
(5.c) $f(x) = \sqrt{1-x} + \sqrt{x-2}$	(5.g) $f(x, r) = \sqrt{r^2 - x^2}$
(5.d) $f(x) = 3$	(5.h) $f(t) = \begin{pmatrix} \sin^2(4t) \\ \cos(5t) \end{pmatrix}$
- Dibujar las curvas definidas por las funciones del Ejercicio 5, utilizando para ello las funciones anónimas allí definidas y la orden `plot`. (Dibujar la función del apartado (5.g) para $r = 5$: $f(5, x)$. Dibujar las dos componentes de la función del apartado (5.h)).
- Representar en la misma ventana gráfica las siguientes funciones, de modo que cada curva tenga un color diferente:

(7.a) $f(x) = 2 \sin^3(x) \cos^2(x)$ y $g(x) = e^x - 2x - 3$, $x \in [-1.5, 1.5]$

$$(7.b) \quad f(x) = \log(x+1) - x \quad y \quad g(x) = 2 - 5x, \quad x \in [0, 5]$$

$$(7.c) \quad f(x) = 6 \operatorname{sen}(x) \quad y \quad g(x) = 6x - x^3, \quad x \in [-\pi/2, \pi/2]$$

$$(7.d) \quad f(x) = e^{-x^2+2} \operatorname{sen}(x/2) \quad y \quad g(x) = -x^3 + 2x + 2, \quad x \in [-1, 2]$$

$$(7.e) \quad f(x) = \sqrt{r^2 - x^2}, \quad \text{para } r = 1 \text{ y } r = 4$$

8. Representar en una misma ventana las funciones siguientes, incluyendo una leyenda y etiquetas en los ejes.

$$(8.a) \quad g(x) = 3^x + 2x - 6 \quad y \quad h(x) = \operatorname{sen}\left(\frac{\sqrt{x^2 + 4x}}{x^3}\right) \quad \text{en el cuadro } [1, 2] \times [0, 1.5]$$

$$(8.b) \quad g(x) = \cos(x^2 + 2x + 1) \quad y \quad h(x) = \log\left(\frac{\sqrt{x+2}}{x^3}\right) \quad \text{en el cuadro } [1, 2] \times [-1.5, 2]$$

$$(8.c) \quad g(x) = \frac{\operatorname{sen}(\pi x)}{1 + \sqrt{2x}} \quad y \quad h(x) = 5e^{-2x} \ln(x^2 + 1) \quad \text{en el cuadro } [0, 2] \times [-0.5, 1]$$

$$(8.d) \quad f(x) = \ln(x^2 - 27) \quad y \quad g(x) = \frac{(x+10)(x-10)(18-x)}{100} \quad \text{en el cuadro } [-15, 20] \times [-20, 15]$$

9. Resolver, si es posible, los siguientes sistemas lineales, comprobando que la solución es correcta:

$$(9.a) \quad \begin{cases} 2x_1 + x_2 - x_3 &= -1 \\ 2x_1 - x_2 + 3x_3 &= -2 \\ 3x_1 - 2x_2 &= 1 \end{cases} \quad (\text{S.C.D.}) \quad (9.d) \quad \begin{cases} x + 2y + 3z &= 2 \\ 2x + y + 3z &= 1 \\ x + y + 2z &= 1 \end{cases} \quad (\text{S.C.I.})$$

$$(9.b) \quad \begin{cases} 2x + 3y &= 8 \\ 3x - y &= -2 \\ -3x + y + z &= 0 \end{cases} \quad (\text{S.C.D.}) \quad (9.e) \quad \begin{cases} 2x + 2y + t &= 1 \\ 2x - 2y + z &= -2 \\ x - z + t &= 0 \\ -4x + 4y - 2z &= 1 \end{cases} \quad (\text{S.I.})$$

$$(9.c) \quad \begin{cases} x_1 + x_2 + x_3 &= 1 \\ 2x_1 - x_2 + x_3 &= 2 \\ x_1 - 2x_2 &= 1 \end{cases} \quad (\text{S.C.I.}) \quad (9.f) \quad \begin{cases} 2x + 3y &= 5 \\ x - y &= 2 \\ 3x + y &= 6 \end{cases} \quad (\text{S.I.})$$

2 Programación con MATLAB

Los condicionales y los bucles o repeticiones son la base de la programación estructurada. Sin ellas, las instrucciones de un programa sólo podrían ejecutarse en el orden en que están escritas (orden secuencial). Las estructuras de control permiten modificar este orden y, en consecuencia, desarrollar estrategias y algoritmos para resolver los problemas.

Los **condicionales** permiten que se ejecuten conjuntos distintos de instrucciones, en función de que se verifique o no determinada condición.

Los **bucles** permiten que se ejecute repetidamente un conjunto de instrucciones, ya sea un número pre-determinado de veces, o bien mientras que se verifique una determinada condición.

2.1 Estructuras condicionales: if

Son los mecanismos de programación que permiten “romper” el flujo secuencial en un programa: es decir, permiten hacer una tarea si se verifica una determinada **condición** y otra distinta si no se verifica.

Evaluar si se verifica o no una condición se traduce, en programación, en averiguar si una determinada **expresión con valor lógico** da como resultado **verdadero** o **falso**

Las estructuras condicionales básicas son las representadas en los diagramas de flujo siguientes:

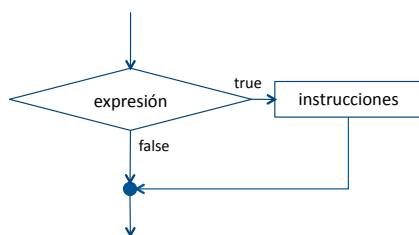


Figura 2.1: Estructura condicional simple: Si **expresión** toma el valor lógico **true**, se ejecutan las instrucciones del bloque **instrucciones** y, después, el programa se continúa ejecutando por la instrucción siguiente. Si, por el contrario, la **expresión** toma el valor lógico **false**, no se ejecutan las **instrucciones**, y el programa continúa directamente por la instrucción siguiente.

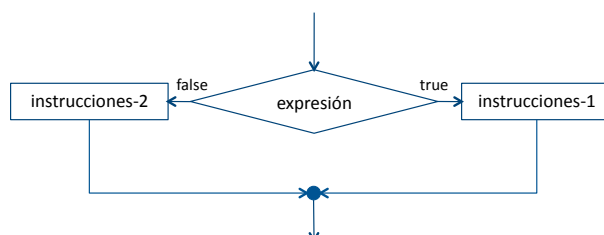


Figura 2.2: Estructura condicional doble: Si **expresión** toma el valor lógico **true**, se ejecutan las instrucciones del bloque **instrucciones 1**. Si, por el contrario, la **expresión** toma el valor lógico **false**, se ejecuta el bloque de **instrucciones 2**. En ambos casos, el programa continúa ejecutándose por la instrucción siguiente.

En casi todos los lenguajes de programación, este tipo de estructuras se implementan mediante una instrucción (o super-instrucción) denominada **if**, cuya sintaxis puede variar ligeramente de unos lenguajes a otros. En MATLAB, concretamente, su forma es la siguiente:

```

instruccion-anterior
if expresion
    bloque-de-instrucciones
end
instruccion-siguiente

```

Esta super-instrucción tiene el funcionamiento siguiente: Se evalúa **expresion**. Si el resultado es **true**, se ejecuta el **bloque-de-instrucciones** y, cuando se termina, se continúa por **instruccion-siguiente**. Si el resultado no es **true**, se va directamente a **instruccion-siguiente**.

```

instruccion-anterior
if expresion
    bloque-de-instrucciones-1
else
    bloque-de-instrucciones-2
end
instruccion-siguiente

```

Su funcionamiento es el siguiente: Se evalúa **expresion**. Si el resultado es **true**, se ejecuta el **bloque-de-instrucciones-1** y, cuando se termina, se continúa por **instruccion-siguiente**. Si el resultado no es **true**, se ejecuta el **bloque-de-instrucciones-2** y cuando se termina se va a la **instruccion-siguiente**.

Ejemplo 2.1 (Uso de un condicional simple)

Escribir una M-función que, dado $x \in \mathbb{R}$, devuelva el valor en x de la función definida a trozos

$$f(x) = \begin{cases} x + 1 & \text{si } x < -1, \\ 1 - x^2 & \text{si } x \geq -1. \end{cases}$$

```

function [fx] = mifun(x)
%
% v = mifun(x) devuelve el valor en x de la función
%           f(x) = x+1    si x < -1
%           f(x) = 1-x^2 si no
%
fx = x + 1;
if x > -1
    fx = 1 - x^2;
end

```

Ejemplo 2.2 (Uso de un condicional doble)

Escribir una M-función que, dados dos números $x, y \in \mathbb{R}$, devuelva un vector cuyas componentes sean los dos números ordenados en orden ascendente.

```

function [v] = Ordena(x, y)
%
% v = Ordena(x, y) es un vector que contiene los dos
%           numeros x e y ordenados en orden creciente
%
if x <= y
    v = [x, y];
else
    v = [y, x];
end

```

Observación. La orden de MATLAB `sort([x,y])` tiene el mismo efecto que esta M-función.

Estas estructuras se pueden “anidar”, es decir, se puede incluir una estructura condicional dentro de uno de los bloques de instrucciones de uno de los casos de otra.

En ocasiones interesa finalizar la ejecución de un programa en algún punto “intermedio” del mismo, es decir, no necesariamente después de la última instrucción que aparece en el código fuente. Esto se hace mediante la orden

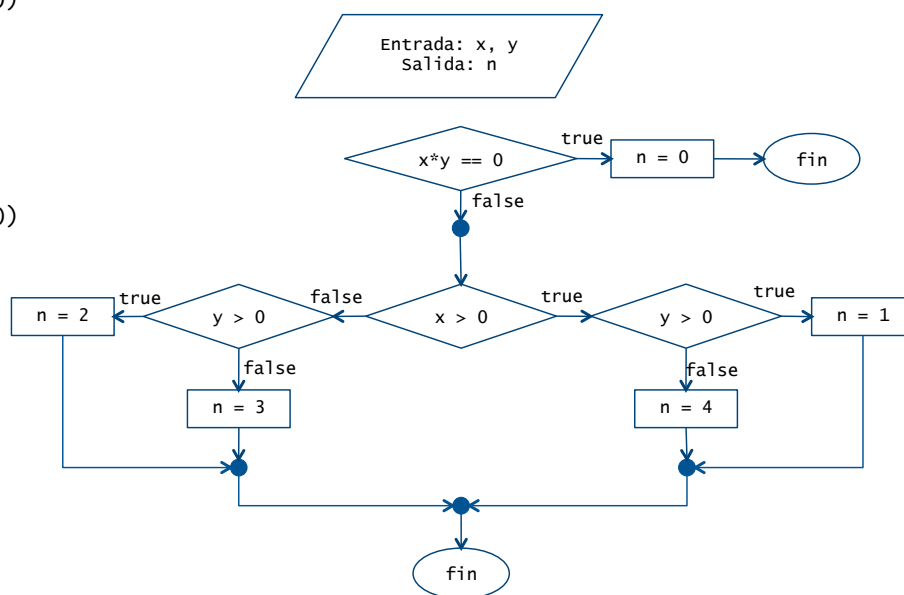
`return`

Ejemplo 2.3 (Condicionales anidados)

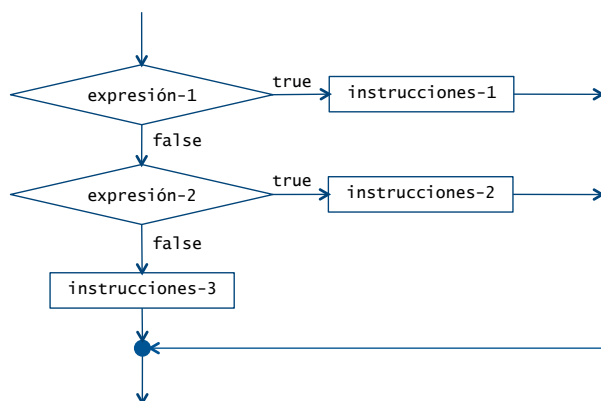
Escribir una M-función que, dados dos números $x, y \in \mathbb{R}$, devuelva el número del cuadrante del plano OXY en el que se encuentra el punto (x, y) . Si el punto (x, y) está sobre uno de los ejes de coordenadas se le asignará el número 0.

```
function [n] = Cuadrante(x, y)
%
% n = Cuadrante(x, y) es el número del cuadrante del plano OXY
%      en el que se encuentra el punto (x,y).
%      n = 0 si el punto está sobre uno de los ejes.
%
if x*y == 0
    n = 0;
    return
end

if (x > 0)
    if (y > 0)
        n=1;
    else
        n=4;
    end
else
    if (y > 0)
        n=2;
    else
        n=3;
    end
end
```



Se pueden construir estructuras condicionales más complejas (con más casos). En MATLAB, estas estructuras se implementan mediante la versión más completa de la instrucción `if`:



```

instruccion-anterior
if expresion-1
    bloque-de-instrucciones-1
elseif expresion-2
    bloque-de-instrucciones-2
else
    bloque-de-instrucciones-3
end
instruccion-siguiente
  
```

Esta estructura tiene el funcionamiento siguiente:

Se evalúa **expresion-1**.

Si el resultado es **true**, se ejecuta el **bloque-de-instrucciones-1** y, cuando se termina, se continúa por **instruccion-siguiente**.

Si el resultado de **expresion-1** no es **true**, se evalúa **expresion-2**.

Si el resultado es **true**, se ejecuta el **bloque-de-instrucciones-2** y, cuando se termina, se continúa por **instruccion-siguiente**.

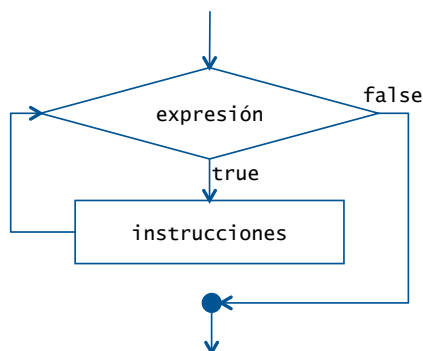
Si el resultado de **expresion-2** no es **true**, se ejecuta el **bloque-de-instrucciones-3** y luego se va a **instruccion-siguiente**.

Es **muy importante** darse cuenta de que, en cualquier caso, se ejecutará **sólo uno** de los bloques de instrucciones.

La cláusula **else** (junto con su correspondiente bloque-de-instrucciones) puede no existir. En este caso es posible que no se ejecute ninguno de los bloques de instrucciones.

2.2 Estructuras de repetición o bucles: while

Este mecanismo de programación permite repetir un grupo de instrucciones **mientras que** se verifique una cierta condición. Su diagrama de flujo y su sintaxis en MATLAB son los siguientes:



```

instruccion-anterior
while expresion
    bloque-de-instrucciones
end
instruccion-siguiente
  
```

Esta estructura tiene el funcionamiento siguiente:

Al comienzo, se evalúa **expresion**.

Si el resultado **no es true**, se va directamente a la **instruccion-siguiente**. En este caso, no se ejecuta el **bloque-de-instrucciones**.

Si, por el contrario, el resultado de **expresion** es **true**, se ejecuta el **bloque-de-instrucciones**. Cuando se termina, se vuelve a evaluar la **expresion** y se vuelve a decidir.

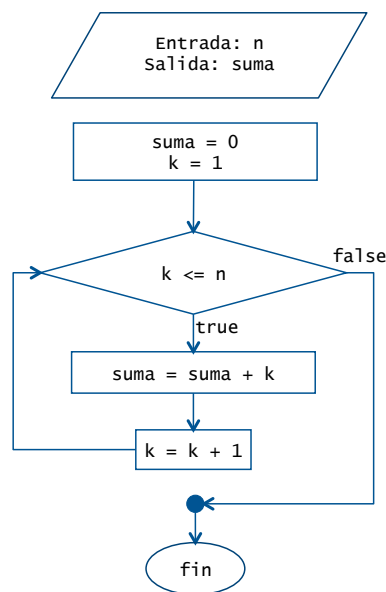
Naturalmente, este mecanismo precisa que, dentro del **bloque-de-instrucciones** se modifique, en alguna de las repeticiones, el resultado de evaluar **expresion**. En caso contrario, el programa entraría

en un *bucle infinito*. Llegado este caso, se puede detener el proceso pulsando la combinación de teclas **CTRL + C**.

Ejemplo 2.4 (Uso de un while)

Escribir una M-función que, dado un número natural n , calcule y devuelva la suma de todos los números naturales entre 1 y n .

```
function [suma] = SumaNat(n)
%
% suma = SumaNat(n) es la suma de los n primeros números naturales
%
suma = 0;
k = 1;
while k <= n
    suma = suma + k;
    k = k + 1;
end
```



Observación. La orden de MATLAB `sum(1:n)` tiene el mismo efecto que `SumaNat(n)`.

Ejercicio.

Partiendo de la M-función `SumaNat` del ejemplo anterior, escribe una M-función que calcule y devuelva la suma de todos los números naturales impares entre 1 y n .

Ejemplo 2.5 (Uso de un while)

Escribir un *script* que genere de forma aleatoria un número natural del 1 a 9 y pida repetidamente al usuario que escriba un número en el teclado hasta que lo acierte.

Observaciones:

- (a) La orden `randi(n)` genera de forma aleatoria un número natural entre 1 y n .
- (b) La orden `var=input('Mensaje')` escribe `Mensaje` en la pantalla, lee un dato del teclado y lo almacena en la variable `var`.
- (c) La orden `beep` emite un sonido.

```
%-----
% Script Adivina
% Este script genera un numero aleatorio entre 1 y 9
% y pide repetidamente que se teclee un numero hasta acertar
%-----
%
n = randi(9);

disp(' ')
disp(' Tienes que acertar un numero del 1 al 9')
disp(' Pulsa CTRL + C si te rindes ...')
disp(' ')

M = 0;
while M~=n
    M=input('Teclea un numero del 1 al 9 : ');
end
beep
disp(' ')
disp('   !!!!Acertaste!!!!   ')
```

Ejercicio.

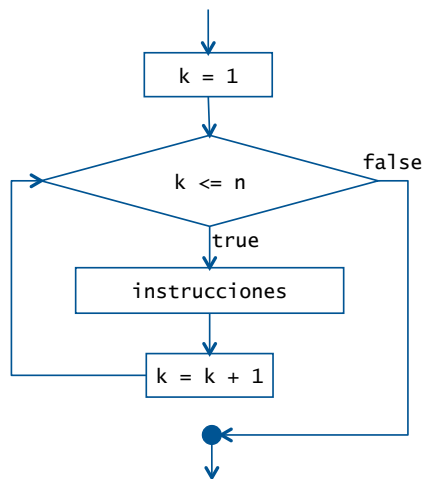
Partiendo del *script* `Adivina` del ejemplo anterior, escribe una M-función que reciba como argumento de entrada un número natural m , genere de forma aleatoria un número natural n entre 1 y m , y pida repetidamente al usuario que escriba un número en el teclado hasta que lo acierte.

2.3 Estructuras de repetición o bucles indexados: for

En muchas ocasiones, las repeticiones de un bucle dependen en realidad de una variable entera cuyo valor se va incrementando hasta llegar a uno dado, momento en que se detienen las repeticiones. Esto sucede, especialmente, con los algoritmos que manipulan vectores y matrices, que es lo más habitual cuando se programan métodos numéricos.

En estas ocasiones, para implementar el bucle es preferible utilizar la estructura que se expone en esta sección: **bucle indexado**. En MATLAB (igual que en muchos otros lenguajes) este tipo de mecanismos se implementan mediante una instrucción denominada **for**.

Por ejemplo, el bucle representado mediante el diagrama siguiente, se puede implementar con las órdenes que se indican:



```

instruccion-anterior
for k = 1 : n
    bloque-de-instrucciones
end
instruccion-siguiente
  
```

Estructura de repetición **for**: Para cada valor de la variable **k** desde **1** hasta **n**, se ejecuta una vez el **bloque-de-instrucciones**. Cuando se termina, el programa se continúa ejecutando por la **instrucción-siguiente**.

Se observa que, con esta instrucción, no hay que ocuparse ni de inicializar ni de incrementar dentro del bucle la variable-índice, **k**. Basta con indicar, junto a la cláusula **for**, el conjunto de valores que debe tomar. Puesto que es la propia instrucción **for** la que gestiona la variable-índice **k**, **está prohibido modificar su valor** dentro del bloque de instrucciones.

El conjunto de valores que debe tomar la variable-índice **k** tiene que ser de números enteros, pero no tiene que ser necesariamente un conjunto de números consecutivos. Son válidos, por ejemplo, los conjuntos siguientes:

```

for k = 0 : 2 : 20           % números pares de 0 a 20
for ind = 30 : -5 : 0       % números 30, 25, 20, 15, 10, 5, 0
for m = [2, 5, 4, 1, 7, 20] % los números especificados
  
```

Observación. Siempre que en un bucle sea posible determinar *a priori* el número de veces que se va a repetir el bloque de instrucciones, es preferible utilizar la instrucción **for**, ya que la instrucción **while** es más lenta.

Ejemplo 2.6 (Uso de for)

Escribir una M-función que, dado un vector **v**, calcule el valor máximo entre todos sus componentes.

```

function [vmax] = Maximo(v)
%
% Maximo(v) es el máximo de las componentes del vector v
%
vmax = v(1);
for k = 2:length(v)
    if v(k) > vmax
        vmax = v(k);
    end
end
  
```

Ejemplo 2.7 (Uso de for)

Observación:

La orden `error('mensaje')` imprime `mensaje` en la pantalla y detiene la ejecución del programa.

Escribir una M-función que calcule la traza de una matriz.

```
function [y] = Traza(A)
%
% Traza(A) es la suma de los elementos diagonales de la
%      matriz cuadrada A
%
[n,m] = size(A);
if n ~= m
    error('La matriz no es cuadrada')
end
y = 0;
for k = 1:n
    y = y + A(k,k);
end
```

2.4 Ruptura de bucles de repetición: break y continue

En ocasiones es necesario interrumpir la ejecución de un bucle de repetición en algún punto interno del bloque de instrucciones que se repiten. Lógicamente, ello dependerá de que se verifique o no alguna condición. Esta interrupción puede hacerse de dos formas:

- Abandonando el bucle de repetición definitivamente.
- Abandonando la iteración en curso, pero comenzando la siguiente.

Las órdenes respectivas en MATLAB son (tanto en un bucle `while` como en un bucle `for`):

```
break
continue
```

El funcionamiento de estas órdenes queda reflejado en los diagramas de flujo correspondientes (Figuras 2.3 y 2.4).

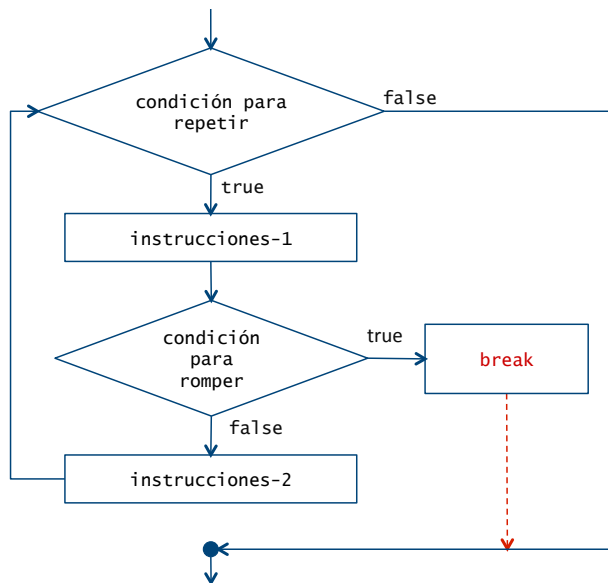


Figura 2.3: Ruptura de bucle **break**: Si en algún momento de un bucle de repetición (de cualquier tipo) se llega a una instrucción **break**, el ciclo de repetición se interrumpe inmediatamente y el programa continúa ejecutándose por la instrucción siguiente a la cláusula **end** del bucle. Si se trata de un bucle indexado **for**, la variable-índice del mismo conserva el último valor que haya tenido.

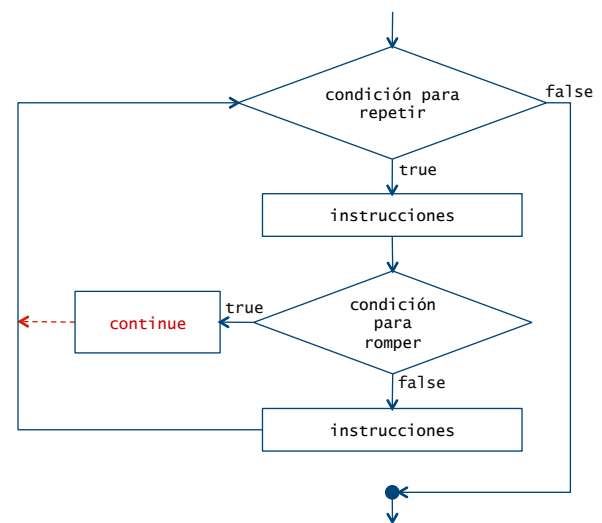


Figura 2.4: Ruptura de bucle **continue**: Si en algún momento de un bucle de repetición (de cualquier tipo) se llega a una instrucción **continue**, la iteración en curso se interrumpe inmediatamente, y se inicia la siguiente iteración (si procede).

2.5 Gestión de errores: warning y error

La instrucción siguiente se utiliza para alertar al usuario de alguna circunstancia no esperada durante la ejecución de un programa, pero no lo detiene. La orden

```
warning('Mensaje')
```

imprime **Warning: Mensaje** en la pantalla y continúa con la ejecución del programa.

Por el contrario, la instrucción

```
error('Mensaje')
```

imprime **??? Mensaje** en la pantalla y detiene en ese punto la ejecución del programa.

Además de estas funciones y sus versiones más completas, MATLAB dispone de otras órdenes para gestionar la detección de errores. Véase **Error Handling** en la documentación.

2.6 Operaciones de lectura y escritura

2.6.1 Instrucción básica de lectura: input

La instrucción `input` permite almacenar en una variable un dato que se introduce a través del teclado. La orden

```
var = input('Mensaje')
```

imprime `Mensaje` en la pantalla y se queda esperando hasta que el usuario teclea algo en el teclado, terminado por la tecla `return`. Lo que se teclea puede ser cualquier expresión que use constantes y/o variables existentes en el `Workspace`. El resultado de esta expresión se almacenará en la variable `var`. Si se pulsa la tecla `return` sin teclear nada se obtendrá una matriz vacía: `[ ]`.

2.6.2 Instrucción básica de impresión en pantalla: disp

La instrucción `disp` permite imprimir en la pantalla el valor de una (matriz) constante o variable, sin imprimir el nombre de la variable o `ans` y sin dejar líneas en blanco. Su utilidad es muy limitada.

```
disp(algo)
```

Ejemplos 2.8 (Uso de disp)

Observaciones:

- (a) La orden `date` devuelve una cadena de caracteres con la fecha actual.
- (b) La orden `num2str(num)` devuelve el dato numérico `num` como una cadena de caracteres.

```
>> disp(pi)
3.1416

>> v = [1, 2, 3, pi];
>> disp(v)
1.0000    2.0000    3.0000    3.1416

>> disp('El metodo no converge')
El metodo no converge

>> disp(['Hoy es ', date])
Hoy es 18-Feb-2015

>> x = pi;
>> disp(['El valor de x es: ', num2str(x)])
El valor de x es: 3.1416
```

2.6.3 Instrucción de impresión en pantalla con formato: fprintf

Esta orden permite controlar la forma en que se imprimen los datos. Su sintaxis para imprimir en la pantalla es

```
fprintf( formato, lista_de_datos )
```

donde:

lista_de_datos son los datos a imprimir. Pueden ser constantes y/o variables, separados por comas.

formato es una cadena de caracteres que describe la forma en que se deben imprimir los datos. Puede contener combinaciones de los siguientes elementos:

- Códigos de conversión: formados por el símbolo %, una letra (como **f**, **e**, **i**, **s**) y eventualmente unos números para indicar el número de espacios que ocupará el dato a imprimir.
- Texto literal a imprimir
- Caracteres de escape, como **\n**.

Normalmente el **formato** es una combinación de texto literal y códigos para escribir datos numéricos, que se van aplicando a los datos de la lista en el orden en que aparecen.

En los ejemplos siguientes se presentan algunos casos simples de utilización de esta instrucción. Para una comprensión más amplia se debe consultar la ayuda y documentación de MATLAB.

Ejemplos 2.9 (Uso de fprintf)

```
>> long = 32.067
>> fprintf('La longitud es de %12.6f metros ',long)
La longitud es de      32.067000 metros
```

En este ejemplo, el formato se compone de:

- el texto literal '**La longitud es de** ' (incluye los espacios en blanco),
- el código **%12.6f** que indica que se escriba un número (en este caso el valor de la variable **long**) ocupando un total de 12 espacios, de los cuales 6 son para las cifras decimales,
- el texto literal ' **metros** ' (también incluyendo los blancos),
- el carácter de escape **\n** que provoca un salto de línea.

```
>> x = sin(pi/5);
>> y = cos(pi/5);
>> fprintf('Las coordenadas del punto son x= %10.6f e y=%7.3f \n', x, y)
Las coordenadas del punto son x=   0.587785 e y=   0.809
```

Observamos que el primer código **%10.6f** se aplica al primer dato a imprimir, **x**, y el segundo código **%7.3f** al segundo, **y**.

```
>> fprintf('En la iteracion k=%3i el error es %15.7e \n', k, err)
En la iteracion k= 23 el error es   3.1406123e-06
```

- el código **%3i** indica que se escriba un número entero ocupando un total de 3 espacios,
- el código **%15.7e** indica que se escriba un número en formato exponencial ocupando un total de 15 espacios, de los cuales 7 son para los dígitos a la derecha del punto decimal.

2.7 Comentarios generales

Algunos comentarios generales sobre la escritura de programas:

- A los argumentos de salida de una M-función **siempre hay que darles algún valor**.
- Las instrucciones **for**, **if**, **while**, **end**, **return**, **...** no necesitan llevar punto y coma al final, ya que no producen salida por pantalla. Sí necesitan llevarlo, en general, las instrucciones incluidas dentro.
- Las M-funciones no necesitan llevar **end** al final. De hecho, a nivel de este curso, que lo lleven puede inducir a error. Mejor no ponerlo.
- Todos los programas deben incluir unas líneas de comentarios que expliquen de forma sucinta su cometido y forma de utilización. En las M-funciones, estas líneas deben ser las siguientes a la instrucción **function**, ya que de esta manera son utilizadas por MATLAB como el texto de **help** de la función. En los *scripts* deben ser las primeras líneas del archivo.
- Es bueno, en la medida de lo posible y razonable, respetar la notación habitual de la formulación de un problema al elegir los nombres de las variables. Ejemplos
 - Llamar **S** a una suma y **P** a un producto, mejor que **G** o **N**.
 - Llamar **i**, **j**, **k**, **l**, **m**, **n** a un número entero, mejor que **x** ó **y**.
 - Llamar **T** o **temp** a una temperatura, mejor que **P**.
 - Llamar **e**, **err** ó **error** a un error, mejor que **vaca**.
- Hay que comprobar los programas que se hacen, dándoles valores a los argumentos para verificar que funciona bien en **todos los casos que puedan darse**. Esto forma parte del proceso de diseño y escritura del programa.
- Los espacios en blanco, en general, hacen el código más legible y por lo tanto más fácil de comprender y corregir. Se deben dejar uno o dos espacios entre el carácter **%** de cada línea de comentario y el texto del comentario (comparéense los dos códigos siguientes).

```
function[Ma,Mg]=Medias(N)
%Ma=Medias(N) devuelve la media aritmetica
%de los numeros naturales menores o iguales
%que N
%[Ma,Me]=Medias(N) devuelve tambien la
media
%geometrica
Ma=0;
Mg=1;
for i=1:N
Ma=Ma+i;
Mg=Mg*i;
end
N1=1/N;
Ma=Ma*N1;
Mg=Mg^(N1);
```

```
function [Ma,Mg]=Medias(N)
%-----
% Ma=Medias(N) devuelve la media
% aritmetica de los numeros
% naturales menores o iguales que N
% [Ma,Me]=Medias(N) devuelve tambien la
% media geometrica
%-----
%
Ma = 0;
Mg = 1;

for i=1:N
    Ma = Ma+i;
    Mg = Mg*i;
end

N1 = 1/N;
Ma = Ma*N1;
Mg = Mg^(N1);
```

2.8 Ejercicios

1. Escribir una M-función `function [N] = Multiplo(k, h)` que reciba como argumentos de entrada dos números enteros k y h y devuelva:
 - $N = 1$ si $k + h$ es múltiplo de 2,
 - $N = 2$ si, además, es múltiplo de 3,
 - $N = 0$ en otro caso.
2. Escribir una M-función de nombre `function [flag] = Divide(m, k, h)` que reciba como argumento de entrada tres números m , k y h , y proporcione como salida un entero `flag` definido como sigue:
 - `flag` = 0 si m no es divisible entre k ,
 - `flag` = 1 si m es divisible entre k ,
 - `flag` = 2 si, además de lo anterior, m es divisible entre h .
3. Escribir una M-función `function [N] = Localizar(x, y)` que devuelva:
 - $N = 1$ si el punto (x, y) está (estrictamente) dentro del círculo de centro $(1, 1)$ y radio $r = 1$,
 - $N = 2$ si, además, está (estrictamente) dentro del círculo de centro $(0, 0)$ y radio $r = 1$,
 - $N = 0$ en otro caso.
4. Escribir una M-función de nombre `function [A] = Areatri(a, b, c)` que calcule el área de un triángulo a partir de las longitudes de sus lados (fórmula de Herón):

$$A = \sqrt{p(p-a)(p-b)(p-c)}, \text{ con } p = \frac{a+b+c}{2}.$$

El programa debe emitir un error con un mensaje explicativo en los casos eventuales en que no se pueda calcular el área: (a) Si alguna de las longitudes es menor que cero; (b) Si el radicando es negativo, lo cual indica que no existe ningún triángulo que tenga esos lados.

5. Escribir una M-función que, dado $x \in \mathbb{R}$, calcule el valor de la función

$$f(x) = \begin{cases} -x & \text{si } x < 0, \\ e^x - 1 & \text{si } x \geq 0. \end{cases}$$

6. Escribir una M-función que, dado $x \in \mathbb{R}$, calcule el valor de la función

$$f(x) = \begin{cases} -x + 2 & \text{si } x < 2, \\ x - 2 & \text{si } x \geq 2. \end{cases}$$

7. Calcular la suma de los 100 primeros números pares (empezando en el 2), utilizando (a) un bucle `for` y (b) un bucle `while`.
8. Calcular e imprimir en pantalla los 10 primeros términos de la sucesión $x_k = \frac{\log(k+1) + 2k}{2k^2 - 1}$
9. Construye un vector con los diez primeros términos de la sucesión

$$\begin{cases} x_1 = 0.25 \\ x_{n+1} = \frac{2x_n - 1}{x_n^2 + 1}, \quad n \geq 1 \end{cases}$$

10. Escribe M-funciones que calculen, usando bucles:

(10.a) `function [vnorm] = Norma(v)`, la norma euclídea del vector v .

(10.b) `function [vmed] = Media(v)`, la media aritmética de los elementos del vector v .

(10.c) `function [vsum] = SumAbs(v)`, la suma de los valores absolutos de los elementos de v .

11. Escribe una M-función `function [u] = ProdAb(A, b, n)` que reciba como datos de entrada una matriz cuadrada A de dimensión n , y un vector b de longitud n , y devuelva el producto Ab calculado elemento a elemento mediante bucles, es decir, sin usar el producto matricial de MATLAB.

12. Escribe una M-función de nombre `function [H] = fhilbert(n)` que reciba como argumento de entrada un entero positivo n y devuelva la matriz de Hilbert de dimensión n , es decir, la matriz definida por

$$H = (h_{ij})_{i,j=1}^n, \quad h_{ij} = \frac{1}{i+j-1}.$$

Estas matrices son un ejemplo notable de mal condicionamiento, incluso para dimensión pequeña.

13. Escribe una M-función `function [w] = SuperaMedia(v)` que reciba como argumento de entrada un vector v y proporcione como salida otro vector w formado por las componentes de v que tengan un valor mayor o igual que la media aritmética de todas las componentes de v . Por ejemplo, si $v=(10,1,7)$, entonces $w=(10,7)$; si $v=(6,5,6,7)$, entonces $w=(6,6,7)$.

14. Escribir una M-función `function [v]=Inter(x,y)` que reciba como argumento de entrada dos vectores fila x e y , y devuelva como salida otro vector v que contenga los elementos comunes a x e y , es decir, la intersección de ambos conjuntos. Se supondrá que ninguno de los vectores x e y tiene elementos repetidos, cosa que no es necesario verificar.

15. Escribir una M-función `function [x] = Bajada(A, b)` para calcular la solución x del sistema $Ax = b$ en el caso en que A es una matriz cuadrada triangular inferior utilizando el **algoritmo de bajada**.

Algoritmo de bajada

n = dimensión de A

Para cada $i = 1, 2, \dots, n$,

$$x_i = \frac{1}{A_{ii}} \left(b_i - \sum_{j=1}^{i-1} A_{ij} x_j \right)$$

Fin

Para comprobar el funcionamiento del programa, construir una matriz 20×20 (por ejemplo) y un vector columna b de números generados aleatoriamente (con la función `rand` o bien con `randi`) y luego extraer su parte triangular inferior con la función `tril`. (Consultar en el help de MATLAB la utilización de estas funciones).

16. Escribir una M-función `function [x] = Subida(A, b)` para calcular la solución x del sistema $Ax = b$ en el caso en que A es una matriz cuadrada triangular superior utilizando el **algoritmo de subida**.

Algoritmo de subida

n = dimensión de A

Para cada $i = n, \dots, 2, 1$

$$x_i = \frac{1}{A_{ii}} \left(b_i - \sum_{j=i+1}^n A_{ij} x_j \right)$$

Fin

Para comprobar el funcionamiento del programa, construir una matriz A y un vector b de números generados aleatoriamente (como en el ejercicio anterior) y luego extraer su parte triangular superior con la función `triu`.

17. La **fórmula de los rectángulos** es una fórmula de integración numérica para aproximar el valor de una integral definida. Dada una partición $a = x_1 < x_2 \dots < x_n = b$ del intervalo $[a, b]$ tal que todos los subintervalos $[x_i, x_{i+1}]$ tienen la misma amplitud, $h = (b - a)/(n - 1)$, la integral definida de f entre a y b se puede aproximar por:

$$\int_a^b f(x) \approx h \sum_{i=1}^{n-1} f(x_i)$$

Utilizando esta fórmula con 15 subintervalos, aproximar el valor de la integral $\int_0^1 e^{-x^2} dx$.

18. La **fórmula de los puntos medios** es una fórmula de integración numérica para aproximar el valor de una integral definida. Dada una partición $a = x_1 < x_2 \dots < x_{n+1} = b$ del intervalo $[a, b]$ tal que todos los subintervalos $[x_i, x_{i+1}]$ tienen la misma amplitud, $h = (b - a)/n$, la integral definida de f entre a y b se puede aproximar por:

$$\int_a^b f(x) \approx h \sum_{i=1}^n f\left(\frac{x_i + x_{i+1}}{2}\right)$$

Escribir una M-función `function [v] = FPuntosmedios(fcn,a,b,n)` que calcule el valor de la integral de `fcn` en `[a,b]` utilizando la fórmula de los puntos medios con `n` subintervalos.

19. La **fórmula de los trapecios** es una fórmula de integración numérica para aproximar el valor de una integral definida. Dada una partición $a = x_1 < x_2 \dots < x_{n+1} = b$ del intervalo $[a, b]$ tal que todos los subintervalos $[x_i, x_{i+1}]$ tienen la misma amplitud, $h = (b - a)/n$, la integral definida de f entre a y b se puede aproximar por:

$$\int_a^b f(x) \approx \frac{h}{2} \left[f(x_1) + 2 \sum_{i=2}^n f(x_i) + f(x_{n+1}) \right]$$

Escribir una M-función `function [v] = Ftrapecios(fcn,a,b,n)` que calcule el valor de la integral de `fcn` en `[a,b]` utilizando la fórmula de los trapecios con `n` subintervalos.

20. La **fórmula de Simpson** es una fórmula de integración numérica para aproximar el valor de una integral definida. Dada una partición $a = x_1 < x_2 \dots < x_{n+1} = b$ del intervalo $[a, b]$ tal que todos los subintervalos $[x_i, x_{i+1}]$ tienen la misma amplitud, $h = (b - a)/n$, la integral definida de f entre a y b se puede aproximar por:

$$\int_a^b f(x) \approx \frac{h}{6} \left[f(x_1) + 2 \sum_{i=2}^n f(x_i) + 4 \sum_{i=2}^{n-1} f\left(\frac{x_i + x_{i+1}}{2}\right) + f(x_{n+1}) \right]$$

Escribir una M-función `function [v] = FSimpson(fcn,a,b,n)` que calcule el valor de la integral de `fcn` en `[a,b]` utilizando la fórmula de Simpson con `n` subintervalos.

3 Resolución de ecuaciones no lineales

Los algoritmos numéricos para resolver el problema

$$\text{Hallar } x \in [a, b] \text{ tal que } f(x) = 0 \quad (3.1)$$

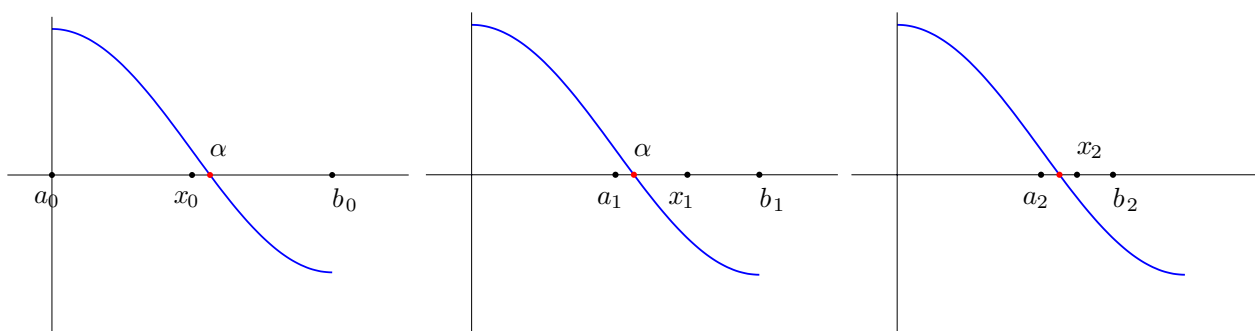
cuando la ecuación $f(x) = 0$ es no lineal (en el caso lineal su resolución es inmediata) son en general **algoritmos iterados**. Esto significa que, a partir de un punto o intervalo iniciales (dependiendo del algoritmo), se construye una sucesión de aproximaciones (calculadas una a partir de la anterior) cuyo límite es la solución buscada. Estos algoritmos, cuando convergen, lo hacen a **una** de las soluciones de la ecuación. Si ésta tuviera más de una solución, habría que utilizar el algoritmo una vez para cada una, cambiando el punto inicial. En la práctica, lógicamente, sólo se efectúan un número finito de iteraciones y el algoritmo se detiene cuando se verifica algún criterio, previamente establecido.

3.1 Métodos iterados y su programación

3.1.1 El método de bisección o dicotomía

Si f es una función continua en el intervalo $[a, b]$ y tiene signos distintos en los extremos, $f(a)f(b) < 0$, entonces posee al menos un cero en el intervalo, es decir, $\exists \alpha \in (a, b)$ tal que $f(\alpha) = 0$. Dicho cero no tiene porqué ser único (salvo que se tengan condiciones adicionales como, por ejemplo, la monotonía estricta de la función).

Una forma sencilla de aproximar el cero de f es dividir el intervalo en dos y elegir el subintervalo en el que se sigue produciendo el cambio de signo. Repitiendo este proceso se puede llegar a conseguir una aproximación de la solución de $f(x) = 0$ tan precisa como se desee.



Algoritmo de dicotomía

- a) Datos de entrada: $a, b, \varepsilon(> 0)$ y $tol(> 0)$.
Hacer $n = 0$ y $e = (b - a)/2$.
- b) Dados n, a, b y e , calcular $x = \frac{a + b}{2}$
- b.1) Si $|e| \leq \varepsilon$ o bien $|f(x)| < tol$, parar y devolver x como aproximación de la solución.
- b.2) Si $f(a)f(x) < 0$, hacer $b = x$.
- b.3) Si no, hacer $a = x$
- c) Hacer $x = \frac{a + b}{2}$, $e = e/2$ y $n = n + 1$ y volver al paso b).

Ejercicio 3.1 (Método de dicotomía) Escribir una M-función que aproxime la solución de $f(x) = 0$ en el intervalo $[a, b]$ utilizando el método de dicotomía.

```
function [x] = Dicotomia(fun, a, b, epsi, tol)
%
% Dicotomia(fun,a,b,eps,tol) devuelve una aproximacion del
% unico cero de la función fun entre a y b
% La función fun debe ser continua y tener signos
% distintos en a y b
%
% Metodo utilizado: biseccion
% Argumentos:
% fun      : un handle a una función
% a, b     : extremos de un intervalo con un cero de fcn
% epsi     : se detiene el algoritmo si |f(x)| < epsi
% tol      : se detiene el algoritmo si |x-sol_exacta| < tol
%
fa = fun(a);
fb = fun(b);
if ( fa*fb > 0 )
    error('La funcion debe cambiar de signo en el intervalo');
end

e = (b-a)*0.5;
x = (a+b)*0.5;

while ( abs(e) >= tol )
    fx = fun(x);
    if ( abs(fx) < epsi )
        return
    end
    if (fa*fx > 0)
        a = x;
```

```
        fa = fx;  
    else  
        b = x;  
        fb = fx;  
    end  
    x = (a+b)*0.5;  
    e = e*0.5;  
end
```

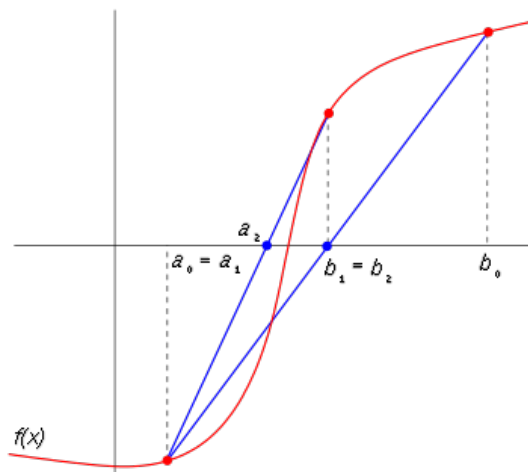
Ejercicio 3.2 Modificar el programa **Dicotomia** de forma que devuelva, además de la aproximación de la solución, el número de iteraciones realizadas:

```
function [x, iter] = Dicotomia(fun,a,b,epsi,tol)
```

Ejercicio 3.3 Modificar el programa **Dicotomia** de forma que imprima una tabla con la evolución de las iteraciones, es decir, con una línea por iteración indicando (a) el número k de la iteración; (b) el valor x_k de la aproximación; (c) el valor $f(x_k)$ de la función en ese punto.

3.1.2 El método de la falsa posición (*regula falsi*)

El método de la falsa posición es una variante del método de bisección consistente también en dividir, en cada etapa, el intervalo en dos y elegir la parte en la que se produce el cambio de signo de la función. Pero, en lugar de dividir el intervalo por su punto medio, se divide por el punto de corte con el eje OX de la secante a la curva $y = f(x)$ que pasa por los puntos $(a, f(a))$ y $(b, f(b))$.



Este método tiene más en cuenta las propiedades de la función f , es capaz de detectar la cercanía del cero a uno de los extremos del intervalo y, en definitiva, cuando el intervalo es pequeño, la secante por los puntos $(a, f(a))$ y $(b, f(b))$ aproxima a la derivada de f . No es extraño, por lo tanto, que se obtengan mejores resultados que con el método de dicotomía.

Algoritmo de Regula Falsi

- a) Datos de entrada: $a, b, \varepsilon(> 0)$ y $tol(> 0)$.
Hacer $n = 0$.
- b) Dados n, a y b , calcular $x = \frac{f(b)a - f(a)b}{f(b) - f(a)}$
 - b.1) Si $|b - a| \leq \varepsilon$ o bien $|f(x)| < tol$, parar y devolver x como aproximación de la solución.
 - b.2) Si $f(a)f(x) < 0$, hacer $b = x$.
 - b.3) Si no, hacer $a = x$
- c) Hacer $x = \frac{f(b)a - f(a)b}{f(b) - f(a)}$ y volver al paso b).

Ejercicio 3.4 (Método de Regula Falsi) Escribir una M-función que aproxime la solución de $f(x) = 0$ en el intervalo $[a, b]$ utilizando el método de la falsa posición.

```
function [x] = RegulaFalsi(fun, a, b, epsi, tol)
%
% RegulaFalsi(fun,a,b,eps,tol) devuelve una aproximacion del
% unico cero de la función fun entre a y b
% La función fun debe ser continua y tener signos
% distintos en a y b
%
% Metodo utilizado: Regula Falsi (Falsa posición)
% Argumentos:
% fun      : un handle a una función
% a,b      : extremos de un intervalo con un cero de fcn
% epsi     : se detiene el algoritmo si |f(x)| < epsi
% tol      : se detiene el algoritmo si |x-sol_exacta| < tol
%
fa = fun(a);
fb = fun(b);
if ( fa*fb > 0 )
    error('La funcion debe cambiar de signo en el intervalo');
end

x = (fb*a-fa*b)/(fb-fa);

while (abs(b-a) > tol )
    fx = fun(x);
    if ( abs(fx) < epsi )
        return
    end
    if ( fa*fx > 0 )
        a = x;
        fa = fx;
    else
        b = x;
        fb = fx;
    end
    x = (fb*a-fa*b)/(fb-fa);
end
```

Ejercicio 3.5 Modificar el programa `RegulaFalsi` de forma que devuelva, además de la aproximación de la solución, el número de iteraciones realizadas:

```
function [x, iter] = RegulaFalsi(fun, a, b, epsi, tol)
```

Ejercicio 3.6 Modificar el programa **RegulaFalsi** de forma que imprima una tabla con la evolución de las iteraciones, es decir, con una línea por iteración indicando (a) el número k de la iteración; (b) el valor x_k de la aproximación; (c) el valor $f(x_k)$ de la función en ese punto.

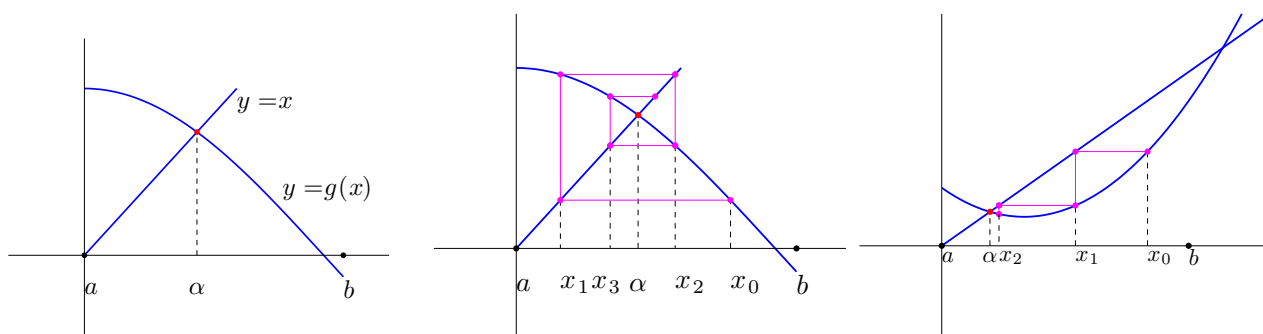
Ejercicio 3.7 Modificar el programa **RegulaFalsi** de forma que para el test de parada se utilice el error relativo en lugar del error absoluto.

3.1.3 El método de aproximaciones sucesivas

Dada $g : [a, b] \subset \mathbb{R} \rightarrow \mathbb{R}$ continua, el método de aproximaciones sucesivas sirve para aproximar soluciones de una **ecuación de punto fijo**

$$x = g(x), \quad x \in [a, b].$$

A las soluciones de $x = g(x)$ se les llama **puntos fijos** de la función g . Geométricamente hallar un punto fijo de g es determinar la abscisa del punto de corte de las gráficas de $y = g(x)$ e $y = x$ en $[a, b]$.



El **método de aproximaciones sucesivas** es un método iterativo que consiste en tomar una aproximación inicial $x_0 \in [a, b]$ y calcular los demás términos de la sucesión $\{x_n\}_{n \geq 0}$ mediante la relación

$$x_{n+1} = g(x_n).$$

Es obvio que cualquier ecuación $f(x) = 0$ puede escribirse en la forma $x = g(x)$ (por ejemplo $x = x + f(x)$, pero también de muchas otras formas). Bajo ciertas condiciones sobre la función g el método de aproximaciones sucesivas es convergente con orden de convergencia lineal (orden 1).

Cuando se utiliza el método de aproximaciones sucesivas para calcular una aproximación del punto fijo, se suelen terminar las iteraciones cuando el valor absoluto de la diferencia entre dos puntos sucesivos sea menor que una tolerancia pre-establecida, ε :

$$|x_n - x_{n-1}| \leq \varepsilon.$$

Por otra parte, para prevenir el caso en que el algoritmo no converja por alguna razón, es preciso imponer que el programa se detenga tras realizar un número máximo prefijado de iteraciones sin que se haya obtenido la convergencia.

Algoritmo de aproximaciones sucesivas (AASS) (para aproximar la solución de $x = g(x)$)

- a) Elegir $x_0 \in [a, b]$ y $\varepsilon > 0$. Hacer $n = 0$.
- b) Dados $n \geq 0$ y x_n .
 - b.1) Calcular $x_{n+1} = g(x_n)$
 - b.2) Si $|x_{n+1} - x_n| \leq \varepsilon$, parar y devolver x_{n+1} como aproximación.
 - b.3) Hacer $n = n + 1$ y repetir el paso b).

Ejercicio 3.8 (Método de aproximaciones sucesivas) Escribir una M-función que aproxime una solución de $x = g(x)$ utilizando el método de aproximaciones sucesivas.

```
function [x] = AASS(fun,xcero,epsi,Nmax)
%
% x = AASS(fun,xcero,tol,Nmax) devuelve una aproximacion de
%      un punto fijo de fun(x), i.e. una solucion de x=fun(x)
%
% Metodo utilizado: aproximaciones sucesivas
% Argumentos:
% xcero      : punto para iniciar el algoritmo
% epsi       : se detiene el algoritmo si |x_{n+1}-x_n| < epsi
% Nmax       : numero maximo de iteraciones a realizar
%
x = xcero;
for k = 1:Nmax
    x0 = x;
    x = fun(x);
    if ( abs(x-x0) < epsi )
        return
    end
end
%
```

Ejercicio 3.9 Modificar el programa **AASS** de forma que devuelva, además de la aproximación de la solución, el número de iteraciones realizadas:

```
function [x, iter] = AASS(fun,xcero,epsi,Nmax)
```

Ejercicio 3.10 Modificar el programa **AASS** de forma que imprima una tabla con la evolución de las iteraciones, es decir, con una línea por iteración indicando (a) el número k de la iteración; (b) el valor x_k de la aproximación; (c) el valor $f(x_k)$ de la función en ese punto.

Ejercicio 3.11 Modificar el programa **AASS** de forma que para el test de parada se utilice el error relativo en lugar del error absoluto.

Ejercicio 3.12 Modificar el programa **AASS** de forma que se produzca un mensaje de *warning* cuando se alcance el número máximo de iteraciones.

3.1.4 El método de Newton

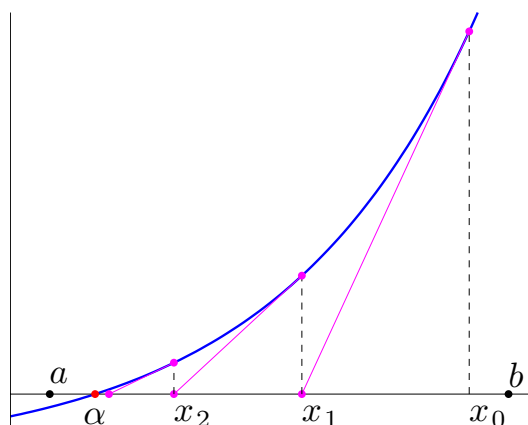
El método de Newton para aproximar la solución de la ecuación

$$f(x) = 0$$

consiste en generar una sucesión $\{x_n\}_{n \geq 0}$ construida a partir de un valor inicial x_0 mediante el método iterado:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Obviamente, el método de Newton necesita del conocimiento de la derivada $f'(x)$ y que esta no se anule en ningún término de la sucesión. La interpretación geométrica del método de Newton es la siguiente: x_{n+1} es la abscisa del punto de intersección con el eje OX de la tangente a la curva $y = f(x)$ en el punto $(x_n, f(x_n))$



Tomando el punto inicial x_0 cercano a la solución, bajo ciertas condiciones sobre la función f , el método de Newton es convergente con orden de convergencia cuadrático (de orden 2).

La forma de detener las iteraciones de este método para obtener una aproximación de la raíz es similar al método de aproximaciones sucesivas:

$$|x_{n-1} - x_n| < \varepsilon.$$

Por otra parte, para prevenir el caso en que el algoritmo no converja por alguna razón, es preciso imponer que el programa se detenga tras realizar un número máximo prefijado de iteraciones sin que se haya obtenido la convergencia.

Algoritmo de Newton (para aproximar la solución de $f(x) = 0$)

- a) Elegir $x_0 \in [a, b]$ y $\varepsilon > 0$.
- b) Dados $n \geq 0$ y x_n .
 - b.1) Hacer $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$.
 - b.2) Si $|x_{n+1} - x_n| < \varepsilon$ o bien $|f(x_{n+1})| < tol$, parar y devolver x_{n+1} como aproximación.

Ejercicio 3.13 Escribir una M-función que aproxime la solución de $f(x) = 0$ utilizando el método de Newton.

```
function [x] = Newton(fcn, dfcn, xcero, epsi, tol, Nmax)
```

fcn: La función que define la ecuación: $f(x) = 0$

dfcn: La derivada $f'(x)$

xcero: El punto inicial de las iteraciones x_0

epsi: El valor de ε para el test de parada $|x_{n+1} - x_n| < \varepsilon$

tol: El valor de tol para detener las iteraciones si $|f(x_{n+1})| < tol$

Nmax: El punto número máximo e iteraciones a realizar

Ejercicio 3.14 Modificar el programa **Newton** de forma que devuelva, además de la aproximación de la solución, el número de iteraciones realizadas:

```
function [x, iter] = Newton(fcn, dfcn, xcero, epsi, tol, Nmax)
```

Ejercicio 3.15 Modificar el programa **Newton** de forma que imprima una tabla con la evolución de las iteraciones, es decir, con una línea por iteración indicando (a) el número k de la iteración; (b) el valor x_k de la aproximación; (c) el valor $f(x_k)$ de la función en ese punto.

Ejercicio 3.16 Modificar el programa **Newton** de forma que para el test de parada se utilice el error relativo en lugar del error absoluto:

$$\frac{|x_{n+1} - x_n|}{|x_n|} < \varepsilon$$

Ejercicio 3.17 Modificar el programa **Newton** de forma que se produzca un *warning* cuando se alcance el número máximo de iteraciones y otro *warning* cuando se llegue a un x_k en que la derivada $f'(x_k)$ sea muy pequeña.

3.2 Resolución de ecuaciones no lineales con MATLAB

3.2.1 La función `fzero`

Para resolver el Problema

$$\text{Hallar } x \in [a, b] \text{ tal que } f(x) = 0 \quad (3.2)$$

MATLAB dispone de la función `fzero`

```
solucion=fzero(funcion,xcero)
```

donde

`funcion` es un manejador de la función que define la ecuación, f . Puede ser el nombre de una **función anónima** dependiente de una sola variable, o también un manejador de una **M-función**, en cuyo caso se escribiría `@funcion`. Ver los ejemplos a continuación.

`xcero` es un valor «cercano» a la solución, a partir del cual el algoritmo iterado de búsqueda de la solución comenzará a trabajar.

`solucion` es el valor (aproximado) de la solución encontrado por el algoritmo.

Ejercicio 3.18 La ecuación siguiente tiene una solución cerca de $x = 1$. Calcular una aproximación.

$$x + \ln\left(\frac{x}{3}\right) = 0$$

1. Definimos una función anónima que evalúe la expresión del primer miembro y usamos el comando `fzero` tomando `x=1` como valor inicial:

```
fun = @(x) x + log(x/3);
fzero(fun,1)
ans =
    1.0499
```

2. Probamos ahora eligiendo un punto inicial más alejado de la solución, por ejemplo:

```
fzero(fun,15)
```

Se recibirá un mensaje diciendo que se aborta la búsqueda de un cero porque durante la búsqueda se han encontrado valores complejos de la función (la función logaritmo no está definida para argumentos negativos en el campo real, pero sí lo está en el campo complejo). En la búsqueda de un intervalo que contenga un cambio de signo de la función, el algoritmo se ha topado con valores de x negativos, en los que la función `log` toma valores complejos.

También podríamos haber usado una M-función para definir la función, en lugar de una función anónima. Mostramos a continuación cómo se haría.

1. Escribimos, en un fichero de nombre `mifuncion.m`, las órdenes siguientes:

```
function [y] = mifuncion(x)
y = x + log(x/3);
```

Salvamos el fichero y lo cerramos. Recuerda que el fichero tiene que llamarse igual que la M-función. En la ventana de comandos, para calcular el cero de la función escribimos:

```
fzero(@mifuncion,1)
```

El algoritmo utilizado por **fzero** comienza por «localizar» un intervalo en el que la función cambie de signo. No funcionará, pues, con ceros en los que no suceda esto, como pasa en el ejemplo siguiente:

Ejercicio 3.19 Utiliza el comando **fzero** para aproximar la solución de la ecuación

$$x^2 = 0$$

Comprobarás que no funciona, sea cual sea el punto inicial.

3.2.2 Gráficas para localizar las raíces y elegir el punto inicial

La determinación teórica de un punto inicial cercano a la solución puede ser una tarea difícil. Si ello es posible, un estudio gráfico previo puede resultar de gran ayuda.

Ejercicio 3.20 Calcular, si existe, una solución positiva de la ecuación $\sin(x) - 2\cos(2x) = 2 - x^2$ determinando un punto inicial a partir de la gráfica de la función.

1. Re-escribimos la ecuación en forma homogénea y representamos gráficamente la función.

```
fun = @(x) sin(x) - 2*cos(2*x) + x.^2 - 2;  
x = linspace(-5,5);  
plot(x,fun(x));
```

La gráfica mostrará que esta función tiene dos ceros: uno positivo cerca de $x = 1$ y otro negativo cerca de $x = -1$.

2. Utilizamos ahora **fzero** para aproximar el cero positivo:

```
fzero(fun,1)  
ans =  
0.8924
```

Como se ha dicho antes, en los casos en que la ecuación tenga varias soluciones, habrá que utilizar **fzero** una vez para cada solución que interese calcular.

Ejercicio 3.21 Calcular las soluciones de la ecuación

$$\sin\left(\frac{x}{2}\right) \cos(\sqrt{x}) = 0 \quad \text{en } [1/2, 3\pi].$$

1. Definimos una función anónima y representamos gráficamente la función:

```
fun = @(x) sin(x/2).*cos(sqrt(x));
x = linspace(0.5,3*pi);
plot(x,fun(x));
grid on
```

Comprobaremos que hay una solución en el intervalo $[2, 3]$ y otra en $[6, 7]$.

2. Utilizamos ahora **fzero** para aproximar cada una de ellas:

```
fzero(fun,2)           % sol. x = 2.4674
fzero(fun,6)           % sol. x = 6.2832
```

o bien

```
fzero(fun,[2,3])
fzero(fun,[6,7])
```

Ejercicio 3.22 Estudiar el número de soluciones de la ecuación siguiente en el intervalo $[-10, 2]$ y calcularlas.

$$(x^2 - 1)e^{x/2} = \frac{(x + 2)^2}{10} - 1$$

Cuando se utiliza la gráfica por ordenador para «localizar» las soluciones de una ecuación es preciso prestar especial atención a los factores de escala de los ejes en el dibujo y hacer sucesivos dibujos «acercándose» a determinadas zonas de la gráfica para comprender la situación.

También será necesario un análisis teórico para no caer en equivocaciones.

Ejercicio 3.23 Calcular todas las soluciones de la ecuación

$$\frac{\ln(x + 1)}{x^2 + 1} = x^2 - 8x + 6.$$

1. Comienza por escribir la ecuación en forma homogénea:

$$\frac{\ln(x + 1)}{x^2 + 1} = x^2 - 8x + 6 \iff f(x) = \ln(x + 1) - (x^2 - 8x + 6)(x^2 + 1) = 0$$

Puesto que no tenemos ninguna indicación del intervalo en el que pueden estar las soluciones, lo primero que hay que observar es que el dominio de definición de la función $f(x)$ es $(-1, +\infty)$.

También conviene analizar cuál es el comportamiento de f en los extremos de este intervalo:

$$\lim_{x \rightarrow (-1)^+} f(x) = -\infty \qquad \lim_{x \rightarrow +\infty} f(x) = -\infty$$

Parece adecuado, pues, comenzar dibujando la gráfica de la función en un intervalo de la forma $[-1, M]$ con M «grande».

Observación: la función $\ln(x+1)$ no está definida para $x = -1$. El cálculo, con MATLAB, de `log(0)` devuelve el valor `-Inf`. La función `plot` ignora los valores `Inf` y `NaN` que aparezcan en el vector `y` de las ordenadas. Por lo tanto, la inclusión del valor $x = -1$ en el intervalo de dibujo no producirá error. Simplemente MATLAB lo ignorará.

- Define una función anónima para f y dibuja su gráfica en el intervalo $[-1, 20]$ (por ejemplo)

```
fun = @(x) log(x+1) - (x.^2-8*x+6).*(x.^2+1);
x = linspace(-1,20);
plot(x,fun(x))
grid on
```

Resulta obvio, a la vista de la gráfica y del análisis del apartado anterior, que no hay ningún cero de f a la derecha de $x = 10$. Sin embargo, entre $x = -1$ y $x = 20$ la situación resulta confusa, ya que la escala del eje OY es demasiado pequeña: observa que la parte visible es $[-1, 20] \times [-10000, 2000]$. Vuelve a dibujar la gráfica, pero ahora en el intervalo (por ejemplo) $[0, 10]$.

- Con esta nueva gráfica parece claro que hay un cero en el intervalo $[0, 2]$ y otro en el intervalo $[6, 8]$. Tratamos de aproximarlos con `fzero`.

```
x1 = fzero(fun,0);           % x1 = 0.7815
x2 = fzero(fun,6);           % x2 = 7.1686
```

Ejercicio 3.24 Las frecuencias naturales de la vibración de una viga homogénea sujeta por un extremo son las soluciones de :

$$f(x) := \cos(x) \cosh(x) + 1 = 0$$

Se desea saber qué raíces tiene f en el intervalo $[0, 15]$. Calcular dichas raíces utilizando la función MATLAB `fzero`.

Realizar las gráficas necesarias para «localizar» los ceros de la función y luego calcularlos, uno a uno, partiendo de un punto suficientemente próximo.

Ejercicio 3.25 Comparar los resultados que se obtienen al calcular (numéricamente) la solución de la ecuación

$$\cos(x) \cosh(x) + 1 = 0$$

en el intervalo $[1, 2]$ utilizando la función `AASS` del Ejercicio 3.6 y la función `fzero`.

4 Integración numérica

4.1 La función *integral* (*quad* en versiones antiguas de MATLAB)

MATLAB dispone de la función `integral` para calcular integrales definidas de funciones reales de variable real¹.

```
integral(fun,a,b);
```

`a, b` son los límites de integración

`fun` es un *handle* de la función a integrar y puede ser especificada de dos formas:

`integral(fun,a,b)` si f está descrita mediante una función anónima

`integral(@fun,a,b)` si f está programada mediante una M-función

La función `integral` utiliza una fórmula de integración adaptativa (i.e. una fórmula en la que los puntos del soporte de integración son *elegidos* de acuerdo con ciertos criterios con el objetivo de minimizar el error) y utiliza parámetros de tolerancia estándar para determinar la precisión (aunque sus valores se pueden elegir utilizando ciertos argumentos opcionales).

En el caso escalar, la función a integrar (anónima o M-función) debe ser escrita de forma vectorizada, esto es, que admita como argumento de entrada un vector y devuelva un vector de la misma dimensión con los valores de la función.

Ejercicio 4.1 Calcular la integral definida $\int_{0.2}^3 x \sin(4 \ln(x)) dx$

```
fun = @(x) x.*sin(4*log(x));  
integral(fun, 0.2, 3)
```

También se podría haber escrito, en una sóla orden:

```
integral(@(x) x.*sin(4*log(x)), 0.2, 3)
```

¹En realidad, la función `integral` permite también calcular (aproximar) integrales impropias y ciertas integrales de línea de funciones de variable compleja, aunque en estas prácticas nos limitamos a ejemplos con funciones reales de una variable real.

Ejercicio 4.2 Calcular la integral definida $\int_0^8 (x e^{-x^{0.8}} + 0.2) dx$ describiendo el integrando mediante una M-función.

Escribe una M-función de nombre (por ejemplo) **mifun**, en un fichero de nombre **mifun.m**:

```
function [y] = mifun(x)
y = x .* exp(-x.^0.8) + 0.2;
```

Después, para calcular la integral, habría que escribir (en la ventana de comandos o en otro programa)

```
integral(@mifun, 0, 8)
```

Ejercicio 4.3 Utilizar la función **integral** para calcular la integral definida

$$\int_{-10}^5 \arctan(x+4) dx$$

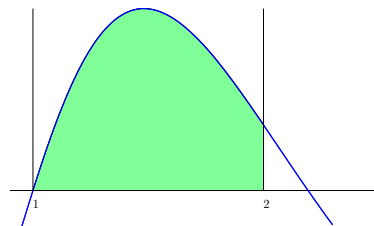
Comparar con el valor exacto de la integral (imprimir los números con todos sus decimales).

(Una primitiva de $f(x) = \arctan(x+4)$ es $F(x) = -\frac{1}{2} \ln(x^2 + 8x + 17) + (x+4) \arctan(x+4)$).

Ejercicio 4.4 Calcular el área de la región plana delimitada por la curva de ecuación $y = \sin(4 \ln(x))$, el eje OX y las rectas verticales $x = 1$ y $x = 2$.

1. Comenzamos analizando y representando gráficamente la función $y = \sin(4 \ln(x))$ para asegurarnos de que es positiva en el intervalo $[1, 2]$:

```
f = @(x) sin(4*log(x));
x1 = linspace(0.95,2.3);
y1=f(x1);
plot(x1,y1)
grid on
```



2. Puesto que f es positiva en $[1, 2]$, el área de la región mencionada es:

$$A = \int_1^2 f(x) dx$$

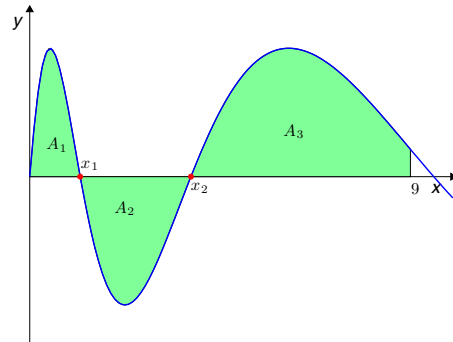
y se calcula con la orden

```
A = integral(f,1,2)
A =
    0.7166
```

Ejercicio 4.5 Calcular el área total determinada por la curva de ecuación $y = \sin(4 \ln(x+1))$ y el eje OX entre los puntos de abscisas $x = 0$ y $x = 9$.

1. Comenzamos por representar gráficamente la función para analizar su signo:

```
f = @(x) sin(4*log(x+1));
x1 = linspace(0,10);
y1=f(x1);
plot(x1,y1)
grid on
```



2. La observación de la gráfica nos muestra que la función f cambia de signo en dos puntos del intervalo $[0, 9]$: x_1 y x_2 y que es positiva en $[0, x_1]$, negativa en $[x_1, x_2]$ y de nuevo positiva en $[x_2, 9]$.

Tenemos que calcular las tres áreas por separado y para ello lo primero es calcular los valores x_1 y x_2 :

```
xcero1 = fzero(f,1);           % xcero1 = 1.1933
xcero2 = fzero(f,4);           % xcero2 = 3.8105
```

3. Calculamos ahora las tres áreas:

$$A_1 = \int_0^{x_1} f(x) dx \quad A_2 = - \int_{x_1}^{x_2} f(x) dx \quad A_3 = \int_{x_2}^9 f(x) dx$$

```
A1 = integral(f,0,xcero1);      % A1 = 0.7514
A2 = - integral(f,xcero1,xcero2); % A2 = 1.6479
A3 = integral(f,xcero2,9);      % A3 = 3.5561
```

4. El área total buscada es finalmente:

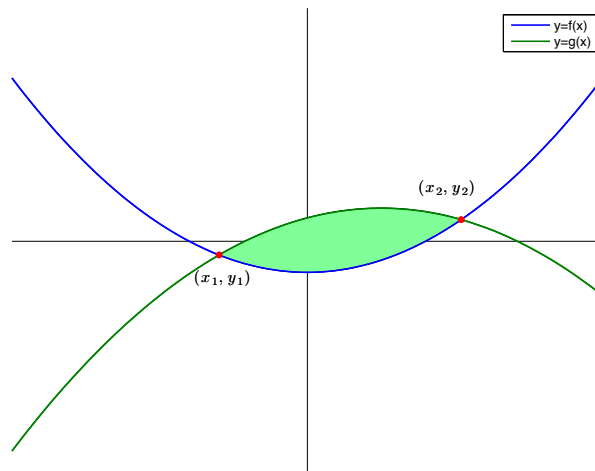
```
A = A1 + A2 + A3;              % A = 5.9554
```

Ejercicio 4.6 Calcular los puntos de corte de las curvas siguientes, así como el área de la región plana encerrada entre ellas

$$y = x^2 - 4 \quad \text{e} \quad y = 2x - 0.8x^2 + 3$$

1. Tenemos que comenzar por identificar la zona en cuestión y hacernos una idea de la localización de los puntos de corte. Para ello vamos a dibujar ambas curvas, por ejemplo, entre $x = -5$ y $x = 5$ (podrían ser otros valores):

```
f = @(x) x.^2-4;
g = @(x) 2*x - 0.8*x.^2 +3;
x = linspace(-5,5);
yf = f(x);
yg = g(x);
plot(x,yf,x,yg)
grid on
```



2. La observación de la gráfica nos muestra que ambas curvas se cortan en dos puntos, (x_1, y_1) y (x_2, y_2) . Para calcularlos comenzamos por calcular sus abscisas x_1 y x_2 :

```
fg = @(x) f(x) - g(x);
x1 = fzero(fg, -1);           % x1 = -1.4932
x2 = fzero(fg, 3);           % x2 = 2.6043
```

Las ordenadas de los puntos de corte son los valores en estos puntos de la función f (o g , ya que toman el mismo valor en ellos):

```
y1 = f(x1);                  % o bien y1 = g(x1);    y1 = -1.7703
y2 = f(x2);                  % o bien y2 = g(x2);    y2 = 2.7826
```

Así pues, las dos curvas se cortan en los puntos $(-1.4932, -1.7703)$ y $(2.6043, 2.7826)$.

3. El área de la región que queda encerrada entre las dos curvas es, ya que en $[x_1, x_2]$ se tiene $g(x) \geq f(x)$:

$$A = \int_{x_1}^{x_2} (g(x) - f(x)) dx = - \int_{x_1}^{x_2} (f(x) - g(x)) dx$$

y se puede calcular con la orden:

```
A = - integral(fg,x1,x2);    % A = 20.6396
```

Ejercicio 4.7 Calcular el área de la región que queda encerrada entre las curvas

$$y = t \operatorname{sen}(\sqrt{5}t) \quad \text{e} \quad y = 0.5 - t.$$

y las rectas verticales $x = 0$ y $x = 10$.

4.2 La función `trapz` (fórmula de los trapecios)

La función MATLAB `trapz` permite calcular, mediante la fórmula de los trapecios, la integral definida de una función definida por un conjunto de datos, es decir, una función de la que sólo se conoce un número finito de puntos de su gráfica:

```
trapz(x,y);
```

`x,y` son dos vectores de la misma dimensión y representan las coordenadas de los puntos. Obviamente, los puntos deben estar ordenados en orden creciente según sus abscisas.

Ejercicio 4.8 Calcular la integral definida entre 0 y 15 de la función discreta dada por el siguiente conjunto de datos:

x	0	2	3	5	6	8	9	11	12	14	15
y	10	20	30	-10	10	10	10.5	15	50	60	85

```
x = [0, 2, 3, 5, 6, 8, 9, 11, 12, 14, 15];
y = [10, 20, 30, -10, 10, 10, 10.5, 15, 50, 60, 85];
trapz(x, y)
```

Ejercicio 4.9 Aproximar la integral del Ejercicio 4.3 mediante la fórmula de los trapecios (función `trapz`), utilizando varios soportes, con distinto número de puntos (5, 15, 30, 100, 1000). Comparar con el resultado obtenido con `integral`. Imprimir los números con todos sus decimales.

Ejercicio 4.10 Se define el promedio integral de una función f en un intervalo $[a, b]$ como:

$$\bar{f} = \frac{1}{b-a} \int_a^b f(x) dx$$

Calcular el promedio integral de la función discreta definida por los datos almacenados en el fichero `discretal.txt`.

5 Interpolación y ajuste polinómico

En Física y otras ciencias con frecuencia es necesario trabajar con conjuntos discretos de valores de alguna magnitud que depende de otra variable. Pueden proceder de muestreos, de experimentos o incluso de cálculos numéricos previos. En ocasiones, para utilizar estos valores en cálculos posteriores es preciso “darles forma” de función, es decir: es preciso disponer de una función dada por una expresión matemática que “coincida” con dichos valores. En Matemáticas también es necesario en ocasiones, sustituir o aproximar una función dada por una más simple o con alguna estructura especial, de forma que ambas tomen los mismos valores en un cierto conjunto de puntos.

En algunas ocasiones, lo anterior no es posible o el resultado no posee las propiedades deseadas. En estos casos se puede recurrir al **ajuste**

5.1 Interpolación global ó de Lagrange

Cuando se trata de encontrar un polinomio (de algún grado determinado) que tome unos valores dados en un conjunto discreto de puntos se habla de **interpolación polinómica global** o **interpolación de Lagrange**. El problema que se plantea es:

Dados un soporte de n puntos distintos $S = \{x_1, x_2, \dots, x_n\}$ con $a \leq x_1 < x_2 < \dots < x_n \leq b$, y n valores $y_1, y_2, \dots, y_n \in \mathbb{R}$, hallar un polinomio de grado $n - 1$

$$p(x) = c_1 x^{n-1} + c_2 x^{n-2} + \dots + c_{n-1} x + c_n$$

que verifique

$$p(x_i) = y_i \quad \forall i = 1, 2, \dots, n$$

Para calcular los coeficientes de este polinomio con MATLAB se usa la orden

```
coef = polyfit(x,y,n-1)
```

x es un vector que contiene los puntos del soporte, x_i .

y es un vector que contiene los valores y_i

n es el número de puntos (i.e. $n - 1$ es el orden del polinomio).

La orden **polyfit** calcula los coeficientes del polinomio de interpolación resolviendo el sistema de Vandermonde.

Una vez calculados los coeficientes, se puede evaluar el polinomio en un punto z mediante la orden

```
pz = polyval(coef,z)
```

coef es un vector que contiene los coeficientes de un polinomio en orden de mayor a menor potencia.

z es el punto en que se desea evaluar el polinomio (puede ser un vector).

pz la salida de esta función es el valor del polinomio en el punto **z** (si **z** es un vector, **pz** será un vector de la misma dimensión que **z**).

Ejercicio 5.1 Calcular el polinomio de interpolación de grado 2 que pasa por los puntos

$$(1, -3), (2, 1), (3, 3)$$

Representar su gráfica en el intervalo $[0, 7]$, señalando con marcadores los puntos interpolados y dibujando también los ejes coordenados. (Escribir un *script* con las órdenes siguientes).

```
%
% Calcular y representar graficamente el polinomio de interpolacion
% global que pasa por tres puntos dados
%
x = [1, 2, 3]
y = [-3, 1, 3];
coef = polyfit(x, y, 2);

z = linspace(0,7);
pz = polyval(coef, z);
plot(z, pz, 'LineWidth',1.1)
hold on
plot([-1,7],[0,0], 'k','LineWidth',1.1)
plot([0,0],[-10,6], 'k','LineWidth',1.1)
plot(x,y,'r.','MarkerSize',20)
axis([-2, 8, -11, 7])
hold off
```

Ejercicio 5.2 Este ejercicio pretende mostrar que el procedimiento de **interpolación global** es, en general inestable, ya que los polinomios tienden a hacerse oscilantes al aumentar su grado y eso puede producir grandes desviaciones sobre los datos (fenómeno de Runge).

Calcula el polinomio de grado 10 que interpola los valores:

```
x=( 0, 2, 3, 5, 6, 8, 9, 11, 12, 14, 15)
y=(10, 20, 30, -10, 10, 10, 10.5, 15, 50, 60, 85)
```

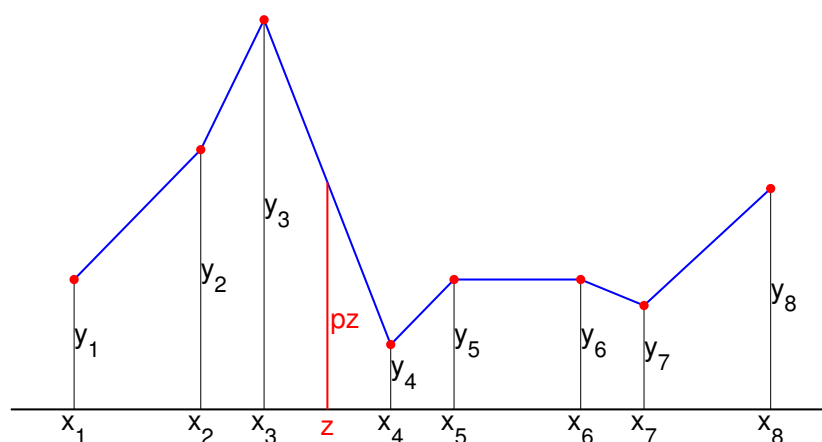
Dibuja su gráfica, así como los puntos con marcadores, y observa las inestabilidades cerca de los extremos. Escribe un *script* o M-función que lleve a cabo todo lo que se pide.

5.2 Interpolación lineal a trozos

Como se ha puesto de manifiesto en el ejercicio 5.2, la interpolación polinómica global es inestable cuando el número de puntos es elevado. En la práctica se usan con frecuencia procedimientos de interpolación polinómica a trozos. En particular, cuando se interpola mediante polinomios de grado 1 a trozos (poligonales), se habla de **interpolación lineal a trozos**.

Dados un soporte de n puntos distintos $S = \{x_1, x_2, \dots, x_n\}$ con $a \leq x_1 < x_2 < \dots < x_n \leq b$, y n valores $y_1, y_2, \dots, y_n \in \mathbb{R}$, el interpolante lineal a trozos asociado es la poligonal representada en la Figura ??, que viene definida, en cada intervalo de la forma $[x_i, x_{i+1}]$ por

$$p_i(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i), \quad i = 1, 2, \dots, n-1$$



Para calcular el valor de la función interpolante en un punto z intermedio entre x_1 y x_n , MATLAB dispone de la función

```
pz = interp1(x,y,z)
```

- x es un vector que contiene los puntos del soporte, x_i .
- y es un vector que contiene los valores y_i
- z es el punto en que se desea evaluar el interpolante (puede ser un vector).
- pz la salida de esta función es el valor del polinomio en el punto z (si z es un vector, pz será un vector de la misma dimensión que z).

Ejercicio 5.3 El fichero de datos `censo.dat` contiene dos columnas que corresponden al censo de EEUU entre los años 1900 y 1990 (en millones de personas).

Representar gráficamente la evolución del censo en esos años y estimar la población que había en el año 1956. (Escribir un *script* con las órdenes siguientes).

```
%
% Interpolacion lineal a trozos de un conjunto de datos
% del censo de los EEUU durante el siglo XX
%

% ----- lectura de los datos
M = load('censo.txt');
anno = M(:,1);
pobl = M(:,2);

% ----- representacion grafica
plot(anno, pobl, 'LineWidth',1.1)
grid on
legend('Evolucion del censo de EEUU')

% ----- poblacion en 1956
p56 = interp1(anno, pobl, 1956);

% ----- inclusion del resultado en la gráfica
texto = [num2str(p56), ' millones de personas'];
text(1910, 230, 'Poblacion en 1956:')
text(1910, 220, texto)
shg
```

Ejercicio 5.4 Se consideran los mismos valores que en el ejercicio 5.2.

Representa gráficamente (juntos) el polinomio de interpolación global y el interpolante lineal a trozos. Representa también los puntos del soporte de interpolación, mediante marcadores. Añade las leyendas adecuadas para que se pueda identificar cada curva adecuadamente.

Calcula y muestra el valor interpolado para $z = 1$ por cada uno de los procedimientos.

Escribe un *script* o M-función que lleve a cabo todo lo que se pide.

Ejercicio 5.5 (para valientes) Escribe tu propia función

```
function [yz] = interpola(x, y, z)
```

que haga lo mismo que `interp1(x, y, z)`.

5.3 Interpolación mediante *splines* cúbicos

La interpolación mediante *splines* busca interpolar un conjunto de datos mediante funciones polinómicas a trozos que no sean sólo continuas, sino que sean continuamente derivables hasta un cierto orden.

Para ello es necesario, naturalmente, aumentar los grados de libertad de que disponemos, es decir, el número de coeficientes que podemos elegir, i.e. el grado de los polinomios.

Concretamente, mediante *splines* cúbicos, se pueden obtener funciones interpolantes de clase C^2 .

El problema ahora es:

Dados un soporte de n puntos distintos $S = \{x_1, x_2, \dots, x_n\}$ con $a \leq x_1 < x_2 < \dots < x_n \leq b$, y n valores $y_1, y_2, \dots, y_n \in \mathbb{R}$, encontrar una función $s \in C^2([a, b])$ que verifique

$$s(x_i) = y_i, \quad i = 1, 2, \dots, n$$

y tal que, restringida a cada subintervalo $[x_i, x_{i+1}]$, sea un polinomio de grado tres.

Para calcular *splines* cúbicos, MATLAB dispone de la función **spline**, que se puede usar de dos formas distintas.

```
pz = spline(x, y, z)
```

x es un vector que contiene los puntos del soporte, x_i .

y es un vector que contiene los valores y_i

z es el punto en que se desea evaluar el *spline* (puede ser un vector).

pz la salida de esta función es el valor del *spline* en el punto **z** (si **z** es un vector, **pz** será un vector de la misma dimensión que **z**).

Otra forma de usar la función **spline** es:

```
coef = spline(x, y)
```

x es un vector que contiene los puntos del soporte, x_i .

y es un vector que contiene los valores y_i

coef es una estructura de datos que contiene los coeficientes que definen el *spline* (no es necesario, a los efectos de este curso, saber más sobre esta estructura)

Una vez calculados los coeficientes del *spline* con esta función, para evaluarlo en un punto z hay que usar la orden

```
pz = ppval(coef, z)
```

coef es la estructura de datos previamente calculada.

z es el punto en que se desea evaluar el *spline* (puede ser un vector).

pz la salida de esta función es el valor del *spline* en el punto **z** (si **z** es un vector, **pz** será un vector de la misma dimensión que **z**).

Ejercicio 5.6 El fichero `DatosSpline.dat` contiene una matriz con dos columnas, que corresponden a las abscisas y las ordenadas de una serie de datos.

Leer los datos del fichero y calcular y dibujar juntos el polinomio de interpolación global y el spline cúbico que interpolan dichos valores, en un intervalo que contenga todos los puntos del soporte. (Escribir un *script* con las órdenes siguientes).

```
% ----- lectura de los datos
datos = load('DatosSpline.txt');
x      = datos(:,1);
y      = datos(:,2);

% ----- calculo de los interpolantes
z = linspace(min(x),max(x));
c = polyfit(x,y,length(x)-1);
p = polyval(c,z);
s = spline(x,y,z);

% ----- graficas
plot(z,p,'b--','LineWidth',1.1);
hold on
plot(z,s,'g','LineWidth',1.1);
plot(x,y,'r.','MarkerSize',15)
legend('Interpolante global', 'Spline cubico', 'Puntos del soporte')
hold off
shg
```

Ejercicio 5.7 En este ejemplo se muestra el uso de la función `spline` en la segunda de las formas que se ha explicado antes: en una primera etapa se calculan los coeficientes del *spline* y se almacenan en una variable, y en una segunda etapa se evalúa el spline en los puntos deseados, utilizando la función `ppval`. Esto permite no repetir el cálculo de los coeficientes (que siempre son los mismos) cada vez que se desea evaluar el *spline*.

Con los datos del mismo fichero `DatosSpline.dat` se pide:

- Definir una función anónima que represente el *spline* cúbico $s(x)$ que interpola dichos valores, es decir, que calcule $s(x)$ para cualquier x .
- Calcular

$$V = \int_0^{40} s(x) dx$$

(Escribir un *script* con las órdenes siguientes).

```
% ----- lectura de los datos
datos = load('DatosSpline.txt');
x      = datos(:,1);
y      = datos(:,2);

% ----- calculo de los coeficientes del spline
coef = spline(x,y);

% ----- definicion de la funcion anonima
s = @(z) ppval(coef, z);

% ----- calculo de la integral
V = integral(s, 0, 40);

% ----- impresion del resultado
fprintf('\n')
fprintf('El valor de la integral es: %15.9f \n',V)
```

Ejercicio 5.8 Se consideran los mismos valores que en el ejercicio 5.2 y el ejercicio 5.4.

Representa gráficamente (juntos) el polinomio de interpolación global, el interpolante lineal a trozos y el spline cúbico. Representa también los puntos del soporte de interpolación, mediante marcadores. Añade las leyendas adecuadas para que se pueda identificar cada curva adecuadamente.

Calcula el valor interpolado para $z = 1$ por cada uno de los procedimientos.

Escribe un *script* o M-función que lleve a cabo todo lo que se pide.

Ejercicio 5.9 (Prescindible. Para ampliar conocimientos) Cuando se calcula un *spline* cúbico con la función **spline** es posible cambiar la forma en que éste se comporta en los extremos. Para ello hay que añadir al vector **y** dos valores extra, uno al principio y otro al final. Estos valores sirven para imponer el valor de la pendiente del spline en el primer punto y en el último. El spline así construido se denomina *sujeto*.

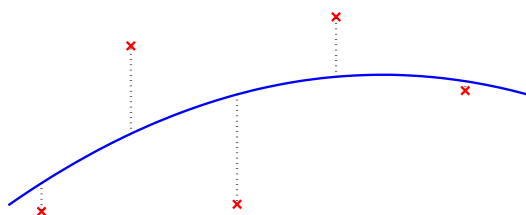
En este ejercicio se trata de calcular y dibujar una aproximación de la función $\sin(x)$ en el intervalo $[0, 10]$ mediante la interpolación con dos tipos distintos de spline cúbico y comparar estos resultados con la propia función, utilizando para ello un soporte regular con 8 puntos. Hay por lo tanto que dibujar tres curvas en $[0, 10]$:

1. La curva $y = \sin(x)$.
2. El spline que calcula MATLAB por defecto (denominado *not-a-knot*).
3. El spline *sujeto* con pendiente $= -1$ en $x = 0$ y pendiente $= 5$ en $x = 10$.

5.4 Mejor aproximación en el sentido de los mínimos cuadrados de un conjunto discreto de puntos mediante polinomios

La técnica de interpolación que hemos explicado antes requiere que la función que interpola los datos pase exactamente por los mismos. En ocasiones esto no da resultados muy satisfactorios, por ejemplo si se trata de muchos datos. También sucede con frecuencia que los datos vienen afectados de algún error, por ejemplo porque provienen de mediciones. No tiene mucho sentido, pues, obligar a la función que se quiere construir a «pasar» por unos puntos que ya de por sí no son exactos.

Otro enfoque diferente es construir una función que no toma exactamente los valores dados, sino que «se les parece» lo más posible, por ejemplo minimizando el error, medido éste de alguna manera.



Cuando lo que se minimiza es la suma de las distancias de los puntos a la curva (medidas como se muestra en la figura) hablamos de **ajuste por mínimos cuadrados**. Veremos solamente cómo se puede hacer esto con MATLAB en algunos casos sencillos.

La función `polyfit` usada ya para calcular el polinomio de interpolación global sirve también para ajustar unos datos por un polinomio de grado dado:

```
coef = polyfit(x, y, m)
```

`x y` son dos vectores de la misma dimensión que contienen respectivamente las abscisas y las ordenadas de los N puntos.

`m` es el grado del polinomio de ajuste deseado

`c` es el vector con los coeficientes del polinomio de ajuste

Si $m = 1$, el polinomio resultante es una recta, conocida con el nombre de **recta de regresión**, si $m = 2$ es una parábola, y así sucesivamente. Naturalmente, cuando $m = N - 1$ el polinomio calculado es el polinomio de interpolación global de grado $N - 1$ que pasa por todos los puntos.

Ejercicio 5.10 Calcula y dibuja (en el intervalo $[0.5, 10]$) los polinomios de ajuste de grado 1, 2, 3 y 6 para los siguientes datos:

```
x = [ 0.9, 1.5, 3, 4, 6, 8, 9.5]
y = [ 0.9, 1.5, 2.5, 5.1, 4.5, 4.9, 6.3]
```

¿Cuál es la ecuación de la recta de regresión?

```
% ----- puntos del soporte
x = [0.9, 1.5, 3, 4, 6, 8, 9.5];
y = [0.9, 1.5, 2.5, 5.1, 4.5, 4.9, 6.3];

% ----- polinomio de grado 1
coef1 = polyfit(x,y,1);
p1 = @(x) polyval(coef1, x);

% ----- polinomio de grado 2
coef2 = polyfit(x,y,2);
p2 = @(x) polyval(coef2, x);

% ----- polinomio de grado 3
coef3 = polyfit(x,y,3);
p3 = @(x) polyval(coef3, x);

% ----- polinomio de grado 6 (pol. interpolacion global)
coef6 = polyfit(x,y,6);
p6 = @(x) polyval(coef6, x);

% ----- graficas
z = linspace(0.5, 10);
plot(z, p1(z), z, p2(z), z, p3(z), z, p6(z))
hold on
plot(x, y, 'm.', 'MarkerSize', 15)
legend('Recta de regresion', 'Parábola de regresion', ...
       'Cubica de regresion','Pol. interpolacion global',...
       'Location', 'SouthEast')
hold off
shg
```

Se obtendrá `coef1 = [0.5688, 0.9982]` luego la recta de regresión es $y = 0.5688x + 0.9982$.

Ejercicio 5.11 En el fichero `Finanzas.dat` está recogido el precio de una determinada acción de la Bolsa española a lo largo de 88 días, tomado al cierre de cada día.

Queremos ajustar estos datos por una recta que nos permita predecir el precio de la acción para un corto intervalo de tiempo más allá de la última cotización.

Lee los datos del fichero y represéntalos mediante una linea poligonal, para observar las oscilaciones de las cotizaciones. Calcula y representa gráficamente la recta de regresión para estos datos.

Ejercicio 5.12 En el fichero `concentracion.dat` se dispone de unos datos, organizados en dos columnas, obtenidos por medición durante una reacción química. La primera columna representa el tiempo, en horas, y la segunda la concentración de cierta sustancia. Se sabe que la concentración, $c(t)$ se comporta como una exponencial: $c(t) = b e^{at}$, por lo que se desea ajustar los datos mediante una curva de ese tipo.

Una forma de encontrar esta función es hacer un cambio de variables que transforme la relación exponencial entre c y t en una relación lineal entre las nuevas variables. Denotamos, pues, $y = \ln(c)$, y se tiene

$$c = b e^{at} \iff y = \ln(c) = \ln(b e^{at}) = \ln(b) + \ln(e^{at}) = \ln(b) + at$$

Así, si ajustamos los datos $(t, y) = (t, \ln(c))$ mediante una recta $y = m + at$, tendremos la curva de ajuste exponencial $c = e^m e^{at}$ para los datos originales.

```
% ----- lectura de los datos
M = load('concentracion.txt');
t = M(:,1);
c = M(:,2);

% ----- calculo de la recta de regresion para los
%           datos t, log(c)
coef = polyfit(t, log(c), 1);

% ----- construccion de la exponencial
a = coef(1);
b = exp( coef(2) );
fc = @(t) b*exp(a*t);

% ----- graficas
plot(t,c,'r. ')
hold on
plot(t, fc(t), 'LineWidth', 1.1)
hold off
shg
```

Ejercicio 5.13 Escribir una M-función

```
function FunInterp(f, a, b, n)
```

que reciba como argumentos de entrada

- **f** : un *handle* de una función
- **a, b** : un intervalo
- **n** : un número entero positivo $n > 2$

La M-función debe construir una partición regular del intervalo $[a, b]$ con n puntos, calcular los valores de la función f en dichos puntos, y calcular y dibujar juntas en el intervalo $[a, b]$ las gráficas de

- los puntos del soporte, con marcadores
 - el polinomio de interpolación global
 - el interpolante lineal a trozos
 - el *spline* cúbico
 - la recta de regresión (ajuste mediante un polinomio de grado 1)
 - la parábola de regresión (ajuste mediante un polinomio de grado 2)
- debidamente identificadas.

6 Resolución de ecuaciones y sistemas diferenciales ordinarios

Una ecuación diferencial ordinaria

$$y' = f(t, y), \quad f: \Omega \subset \mathbb{R}^2 \rightarrow \mathbb{R}$$

admite, en general, infinitas soluciones. Si, por ejemplo, $f \in C^1(\Omega; \mathbb{R})$ ¹, por cada punto $(t_0, y_0) \in \Omega$ pasa una única solución $\varphi: I \rightarrow \mathbb{R}$, definida en un cierto intervalo $I \subset \mathbb{R}$, que contiene a t_0 .

Se denomina Problema de Cauchy (PC) o Problema de Valor Inicial (PVI)

$$\begin{cases} y' = f(t, y), \\ y(t_0) = y_0 \end{cases} \quad (6.1)$$

al problema de determinar, de entre todas las soluciones de la ecuación diferencial $y' = f(t, y)$, aquella que pasa por el punto (t_0, y_0) , es decir, que verifica $y(t_0) = y_0$.

Sólo para algunos (pocos) tipos muy especiales de ecuaciones diferenciales es posible encontrar la expresión de sus soluciones en términos de funciones elementales. En la inmensa mayoría de los casos prácticos sólo es posible encontrar aproximaciones numéricas de los valores de una solución en algunos puntos.

Así, una **aproximación numérica** de la solución del problema de Cauchy

$$\begin{cases} y' = f(t, y), & t \in [t_0, t_f] \\ y(t_0) = y_0 \end{cases} \quad (6.2)$$

consiste en una sucesión de valores de la variable independiente:

$$t_0 < t_1 < \dots < t_n = t_f$$

y una sucesión de valores $y_0, y_1, \dots, y_n$ tales que

$$y_k \approx y(t_k), \quad k = 0, 1, \dots, n,$$

es decir, y_k es una aproximación del valor en t_k de la solución del problema (6.2).

$$y_k \approx y(t_k), \quad k = 0, 1, \dots$$

¹En realidad basta con que f sea continua y localmente Lipschitziana con respecto de y en Ω .

6.1 Resolución de problemas de Cauchy con MATLAB

MATLAB dispone de toda una familia de funciones para resolver (numéricamente) ecuaciones diferenciales:

```
ode45, ode23, ode113
ode15s, ode23s, ode23t, . . .
```

Cada una de ellas implementa un método numérico diferente, siendo adecuado usar unas u otras en función de las dificultades de cada problema en concreto. Para problemas no demasiado «difíciles» será suficiente con la función **ode45** ó bien **ode23**.

Exponemos aquí la utilización de la función **ode45**, aunque la utilización de todas ellas es similar, al menos en lo más básico. Para más detalles, consúltese la documentación.

Para dibujar la gráfica de la solución (numérica) de (6.2) se usará la orden:

```
ode45(odefun, [t0,tf], y0)
```

donde:

odefun es un manejador de la función que evalúa el segundo miembro de la ecuación, $f(t, y)$. Puede ser el nombre de una función anónima dependiente de dos variables, siendo t la primera de ellas y y la segunda, o también una M-función, en cuyo caso se escribiría **@odefun**. Ver los ejemplos a continuación.

[t0,tf] es el intervalo en el que se quiere resolver la ecuación, i.e. **t0** = t_0 , **tf** = t_f .

y0 es el valor de la condición inicial, **y0** = y_0 .

Ejercicio 6.1 Calcular la solución del Problema de Cauchy:

$$\begin{cases} y' = 5y & \text{en } [0, 1] \\ y(0) = 1 \end{cases}$$

Comparar (gráficamente) con la solución exacta de este problema, $y = e^{5t}$

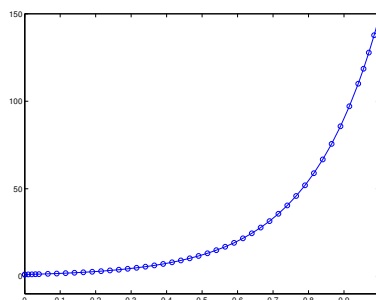
1. Comenzamos por definir una función anónima que represente el segundo miembro de la ecuación $f(t, y)$:

```
f = @(t,y) 5*y;
```

2. Calculamos y dibujamos la solución numérica:

```
ode45(f, [0,1], 1)
```

Obsérvese que no se obtiene ningún resultado numérico en salida, únicamente la gráfica de la solución.



3. Para comparar con la gráfica de la solución exacta, dibujamos en la misma ventana, sin borrar la anterior:

```
hold on
t = linspace(0,1);
plot(t, exp(5*t), 'r')
shg
```

4. Para hacer lo mismo utilizando una M-función en vez de una función anónima, tendríamos que escribir, en un fichero (el nombre `odefun` es sólo un ejemplo):

```
function [dy] = odefun(t,y)
% Segundo miembro de la ecuacion diferencial
dy = 5*y;
```

y guardar este texto en un fichero de nombre `odefun.m` (el mismo nombre que la función). Este fichero debe estar en la carpeta de trabajo de MATLAB.

Después, para resolver la ecuación, usaríamos `ode45` en la forma:

```
ode45(@odefun, [0,1], 1)
```

Si se desean conseguir los valores t_k e y_k de la aproximación numérica para usos posteriores, se debe usar la función `ode45` en la forma:

```
[t, y] = ode45(odefun, [t0,tf], y0);
```

obteniéndose así los vectores **t** e **y** de la misma longitud que proporcionan la aproximación de la solución del problema:

$$y(k) \approx y(t(k)), \quad k = 1, 2, \dots, \text{length}(y).$$

Si se utiliza la función `ode45` en esta forma, **no se obtiene ninguna gráfica**.

Ejercicio 6.2 Calcular el valor en $t = 0.632$ de la solución del problema

$$\begin{cases} y' = t e^{t/y} \\ y(0) = 1 \end{cases}$$

Calculamos la solución en el intervalo $[0, 1]$ y luego calculamos el valor en $t = 0.632$ utilizando una interpolación lineal a trozos:

```
f      = @(t,y) t.*exp(t/y);
[t,y] = ode45(f, [0,1], 1);
v      = interp1(t, y, 0.632)
```

Obsérvese que **t** e **y**, las variables de salida de **ode45**, son dos vectores columna de la misma longitud.

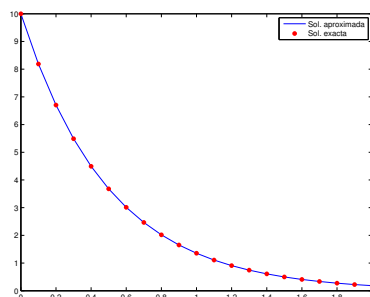
En ocasiones puede ser necesario calcular las aproximaciones y_k en unos puntos t_k pre-determinados. Para ello, en lugar de proporcionar a **ode45** el intervalo $[t_0, t_f]$, se le puede proporcionar un vector con dichos valores, como en el ejemplo siguiente:

Ejercicio 6.3 Calcular (aproximaciones de) los valores de la solución del problema

$$\begin{cases} y' = -2y \\ y(0) = 10 \end{cases}$$

en los puntos: 0, 0.1, 0.2, ..., 1.9, 2. Comparar (gráficamente) con la solución exacta $y = 10e^{-2t}$.

```
f = @(t,y) -2*y;
t = 0:0.1:2;
[t,y] = ode45(f, t, 10);
plot(t, y, 'LineWidth', 1.1)
hold on
plot(t, 10*exp(-2*t), 'r.')
legend('Sol. aproximada', 'Sol. exacta')
shg
```



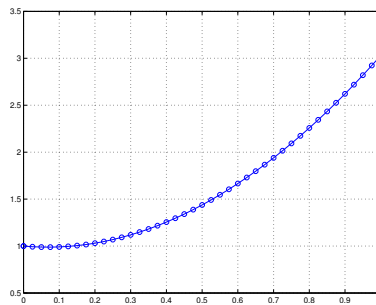
Ejercicio 6.4 Calcular el instante en que la solución del problema

$$\begin{cases} y' = 0.5(10t - \ln(y + 1)) \\ y(0) = 1 \end{cases}$$

alcanza el valor $y = 1.5$.

1. Comenzamos por visualizar la gráfica de la solución, para obtener, por inspección, una primera aproximación:

```
f = @(t,y) 0.5*(10*t-log(y+1));
ode45(f, [0,1], 1)
grid on
shg
```



Vemos así que el valor $y = 1.5$ se produce para un valor de t en el intervalo $[0.5, 0.6]$.

2. Usaremos ahora la función `fzero` para calcular la solución de la ecuación $y(t) = 1.5$ escrita en forma homogénea, es decir, $y(t) - 1.5 = 0$, dando como aproximación inicial (por ejemplo) $t = 0.5$. Para ello, necesitamos construir una función que interpole los valores que nos devuelva `ode45`. Usamos interpolación lineal a trozos:

```
[ts,ys] = ode45(f, [0,1], 1);
fun = @(t) interp1(ts, ys, t) - 1.5;
fzero(fun, 0.5)
```

Obtendremos el valor $t \approx 0.5292$.

